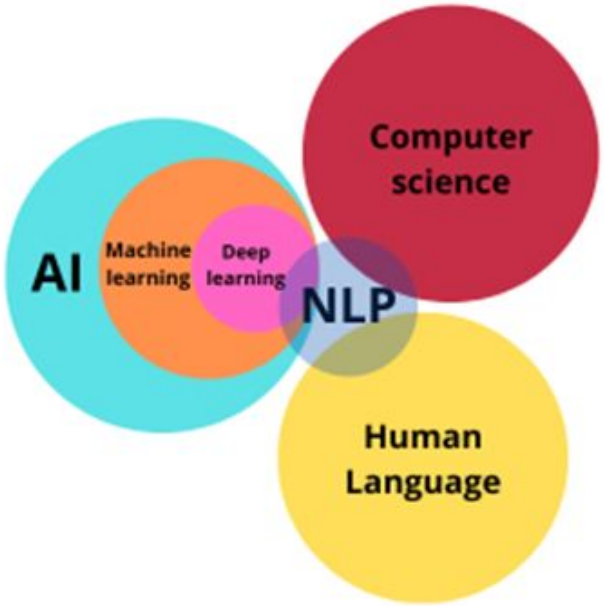
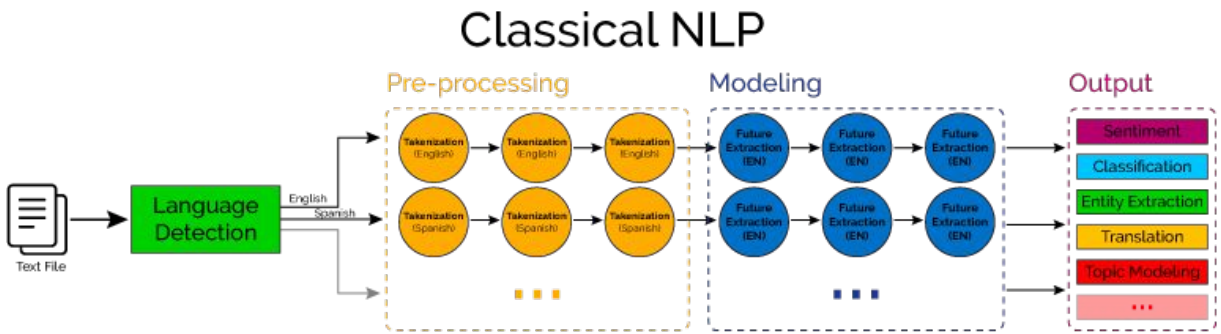
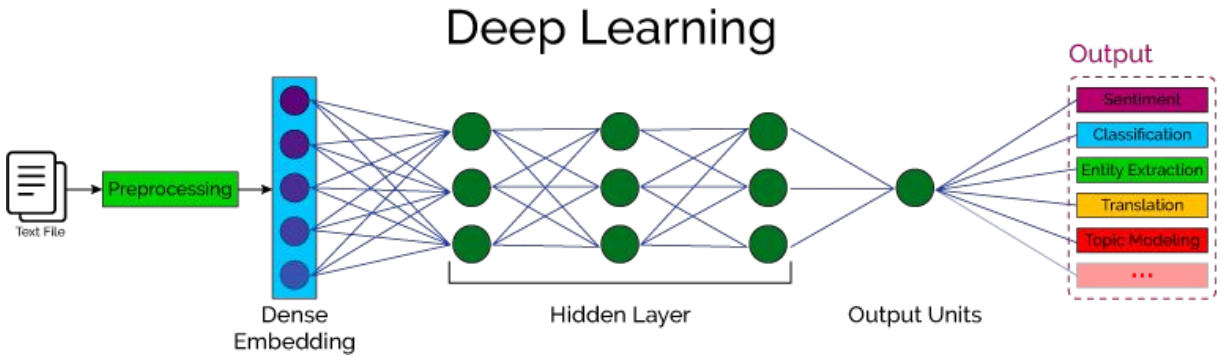
Natural Language Processing







Natural Language Processing

The Bag of Words Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!

it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
…	…

Bag of words

```
corpus = [
    "This is the best book I have ever read",
    "Reading books is the best way to gain knowledge",
    "Which book you prefer. This one or the another one?",
    "Is this the book you love most?"
]
```

Make vocabulary

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
another	best	book	books	ever	gain	have	is	knowledge	love	most	one	or	prefer	read	reading	the	this	to	way	which	you

Score assignment

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
0	1	1	0	1	0	1	1	0	0	0	0	0	0	1	0	1	1	0	0	0	0
0	1	0	1	0	1	0	1	1	0	0	0	0	0	0	1	1	0	1	1	0	0
1	0	1	0	0	0	0	0	0	0	0	2	1	1	0	0	1	1	0	0	1	1
0	0	1	0	0	0	0	1	0	1	1	0	0	0	0	0	1	1	0	0	0	1

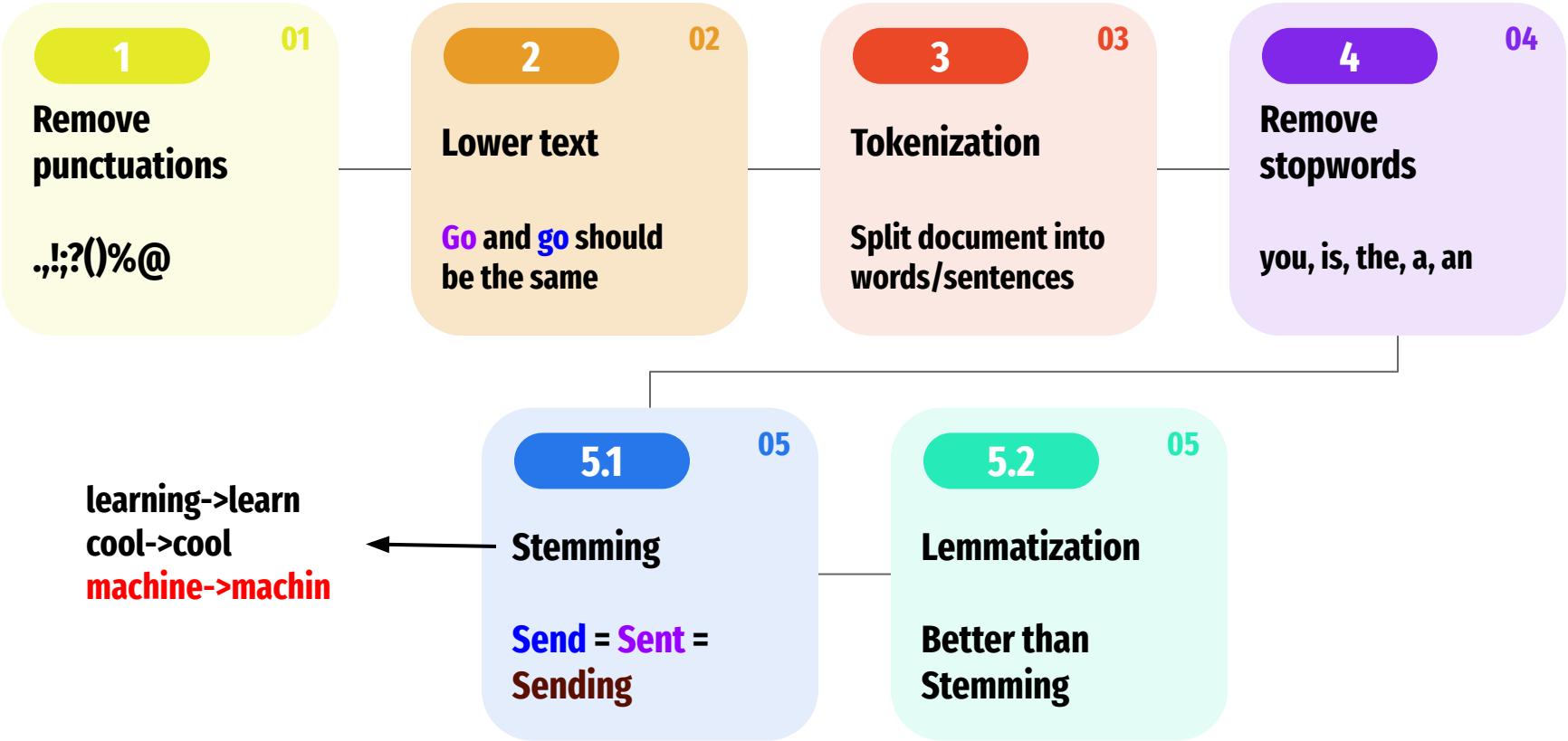
CountVectorizer

```python
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "This is the best book I have ever read",
    "Reading books is the best way to gain knowledge",
    "Which book you prefer. This one or the another one?",
    "Is this the book you love most?"
]
vectorizer = CountVectorizer()
data = vectorizer.fit_transform(corpus)
vocabulary = vectorizer.get_feature_names_out()
print("Vocabulary:")
print(vocabulary)
result = data.toarray()
print("Bag of words:")
print(result)
```

example

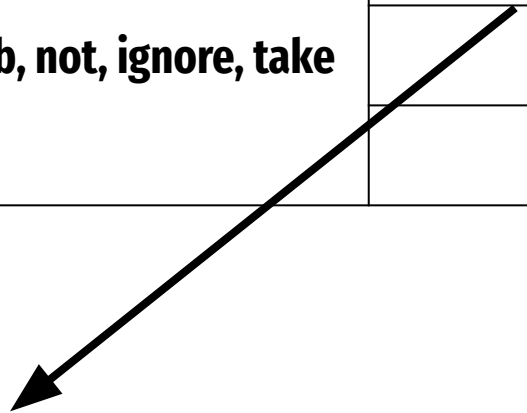
```
C:\Users\Viet\anaconda3\envs\basic\python.exe C:\Users\Viet\PycharmProjects\basic\example.py
Vocabulary:
['another' 'best' 'book' 'books' 'ever' 'gain' 'have' 'is' 'knowledge'
 'love' 'most' 'one' 'or' 'prefer' 'read' 'reading' 'the' 'this' 'to'
 'way' 'which' 'you']
Bag of words:
[[0 1 1 0 1 0 1 1 0 0 0 0 0 0 1 0 1 1 0 0 0 0]
 [0 1 0 1 0 1 0 1 1 0 0 0 0 0 0 1 1 0 1 1 0 0]
 [1 0 1 0 0 0 0 0 0 0 0 2 1 1 0 0 1 1 0 0 1 1]
 [0 0 1 0 0 0 0 1 0 1 1 0 0 0 0 0 1 1 0 0 0 1]]
```

Text preprocessing



Bag of words

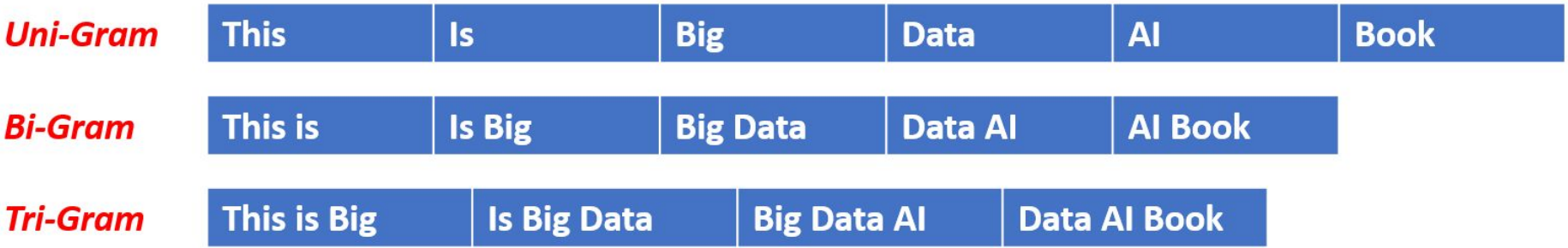
Document	Vocabulary (no stopwords)	Bag of words' vector
This is a good job. I will not ignore it	Good, job, not, ignore, take	1,1,1,1,0
This is not a good job. I will ignore it		1,1,1,1,0
This is a good job. I will take it		1,1,0,0,1



N-grams

N-grams

This is Big Data AI Book



CountVectorizer

```python
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "This is the best book I have ever read",
    "Reading books is the best way to gain knowledge",
    "Which book you prefer. This one or the another one?",
    "Is this the book you love most?"
]
vectorizer = CountVectorizer(ngram_range=(1, 2), stop_words="english")
```

BoW

```
Vocabulary:
['another' 'another one' 'best' 'best book' 'best way' 'book' 'book have'
 'book you' 'books' 'books is' 'ever' 'ever read' 'gain' 'gain knowledge'
 'have' 'have ever' 'is' 'is the' 'is this' 'knowledge' 'love' 'love most'
 'most' 'one' 'one or' 'or' 'or the' 'prefer' 'prefer this' 'read'
 'reading' 'reading books' 'the' 'the another' 'the best' 'the book'
 'this' 'this is' 'this one' 'this the' 'to' 'to gain' 'way' 'way to'
 'which' 'which book' 'you' 'you love' 'you prefer']
Bag of words:
[[0 0 1 1 0 1 1 0 0 0 1 1 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0
  1 1 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 1 0 1 0 0 0 1 1 0 0 1 1 0 0 1 1 0 1 0 0 0 0 0 0 0 0 0 1 1 1 0 1 0
  0 0 0 0 1 1 1 1 0 0 0 0 0]
 [1 1 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 1 1 1 1 1 0 0 0 1 1 0 0
  1 0 1 0 0 0 0 0 1 1 1 0 1]
 [0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 1 0 1 0 1 1 1 0 0 0 0 0 0 0 0 0 1 0 0 1
  1 0 0 1 0 0 0 0 0 0 1 1 0]]
```

Term Frequency-Inverse Document Frequency

$$TF = \frac{Number\ of\ times\ a\ word\ "X"\ appears\ in\ a\ Document}{Number\ of\ words\ present\ in\ a\ Document}$$

$$IDF = log\left(\frac{Number\ of\ Documents\ present\ in\ a\ Corpus}{Number\ of\ Documents\ where\ word\ "X"\ has\ appeared}\right)$$

$$TF\ IDF\ =\ TF * IDF$$

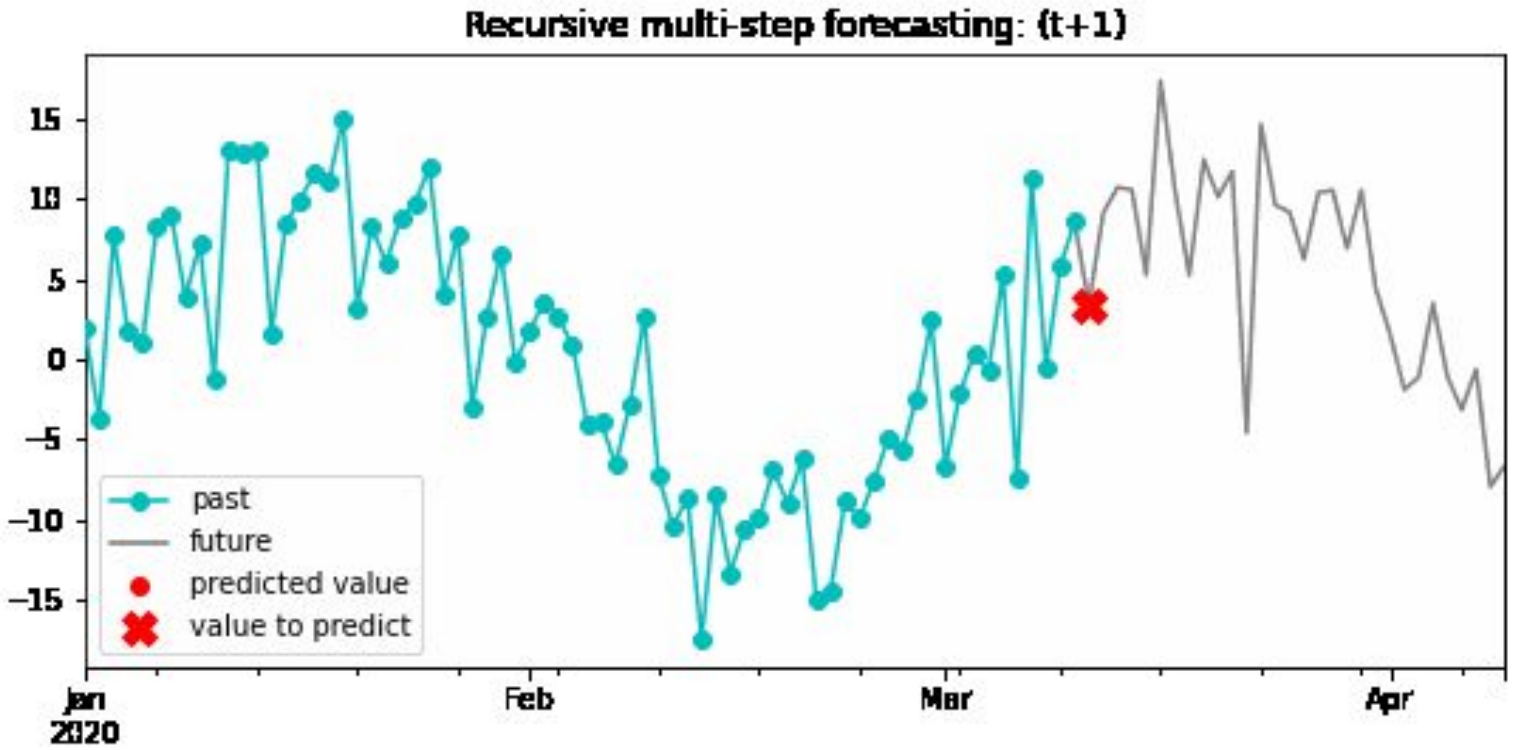
Term Frequency (TF)

Tần số xuất hiện của 1 word trong 1 document

Inverse Document Frequency (IDF)

Mức độ phổ biến/hiếm của 1 word trong toàn bộ các documents

Time series forecasting



Time series data

date	charter_foreign	total_foreign	charter_domestic	total_domestic
date	bigint	bigint	bigint	bigint
Date	Integer	Integer	Integer	Integer
2008-01-01T00:00:00.000Z	57463	5642057	174692	7394181
2008-02-01T00:00:00.000Z	56651	5053435	142931	6837201
2008-03-01T00:00:00.000Z	95878	6071551	176906	8439191
2008-04-01T00:00:00.000Z	100198	5731709	103054	7467710
2008-05-01T00:00:00.000Z	130720	6003335	88631	7840260
2008-06-01T00:00:00.000Z	196656	6467543	99579	8274573
2008-07-01T00:00:00.000Z	195873	6861571	130766	8971734
2008-08-01T00:00:00.000Z	152848	6895232	77253	8717085
2008-09-01T00:00:00.000Z	88267	5539617	40070	6371669
2008-10-01T00:00:00.000Z	76723	5564387	43003	6520788
2008-11-01T00:00:00.000Z	26671	5128350	50322	6223857
2008-12-01T00:00:00.000Z	30963	5624131	54894	6959043
2009-01-01T00:00:00.000Z	35174	5399479	60652	6805171

	Jan-17	Feb-17	Mar-17	Apr-17	May-17	Jun-17	Jul-17	Aug-17	Sep-17	Oct-17	Nov-17	Dec-17
Store 1	20 809	23 832	22 542	18 736	15 347	12 190	11 049	10 913	9 019	9 803	3 172	7 768
Store 2	14 834	14 170	13 109	7 560	6 388	8 124	6 719	3 847	3 187	8 145	10 007	10 036
Store 3	14 453	12 816	14 260	12 954	12 596	11 200	8 374	9 725	10 853	11 609	13 897	19 628
Store 4	8 590	8 822	13 497	13 073	13 278	14 705	13 550	13 962	14 783	11 712	13 234	17 790
Store 5	16 647	12 851	13 433	13 486	13 767	10 482	11 476	13 024	14 459	14 672	12 885	16 681
Store 6	7 323	5 043	4 857	2 584	6 990	4 130	45 110	4 895	5 708	5 106	4 334	10 741
Store 7	8 735	4 645	4 905	4 536	5 385	5 040	6 341	9 403	9 904	9 521	13 473	7 077
Store 8	965	1 254	1 937	1 940	1 756	1 690	1 536	2 502	2 806	4 087	3 704	5 478
Store 9	188	4 375	5 109	5 255	5 667	6 331	9 182	9 867	8 948	10 096	10 860	10 951
Store 10	14 033	30 900	18 864	12 828	10 269	8 464	7 674	4 160	5 497	9 608	16 177	15 798
Store 11	8 185	9 090	12 560	13 682	10 061	8 172	8 045	6 622	7 240	9 923	9 403	10 768
Store 12	911	384	658	949	1 236	1 405	1 366	1 433	1 282	1 515	1 457	2 933
Store 13	2 361	2 830	2 892	2 953	2 929	2 982	6 361	25 969	22 952	27 171	27 199	35 298
Store 14	9 805	8 436	12 878	9 580	6 665	5 069	6 003	6 139	10 987	11 039	11 252	13 844

Code	Name	Population|1990	Population|2000	Population|2010	Population Change|1980-1990	Population Change|1990-2000
AK	Alaska	550043	626932	710231	36.9	14
AL	Alabama	4040587	4447100	4779736	3.8	10.1
AR	Arkansas	2350725	2673400	2915918	2.8	13.7
AZ	Arizona	3665228	5130632	6392017	34.8	40
CA	California	29760021	33871648	37253956	25.7	13.8
CO	Colorado	3294394	4301261	5029196	14	30.6
CT	Connecticut	3287116	3405565	3574097	5.8	3.6
DC	District of Columbia	606900	572059	601723	-4.9	-5.7
DE	Delaware	666168	783600	897934	12.1	17.6
FL	Florida	12937926	15982378	18801310	32.7	23.5
GA	Georgia	6478216	8186453	9687653	18.6	26.4
HI	Hawaii	1108229	1211537	1360301	14.9	9.3
IA	Iowa	2776755	2926324	3046355	-4.7	5.4
ID	Idaho	1006749	1293953	1567582	6.7	28.5
IL	Illinois	11430602	12419293	12830632	0	8.6
IN	Indiana	5544159	6080485	6483802	1	9.7
KS	Kansas	2477574	2688418	2853118	4.8	8.5
KY	Kentucky	3685296	4041769	4339367	0.7	9.7
LA	Louisiana	4219973	4468976	4533372	0.3	5.9

Time series data

1	2	3	4	5	6	7	8	9	10

Time series

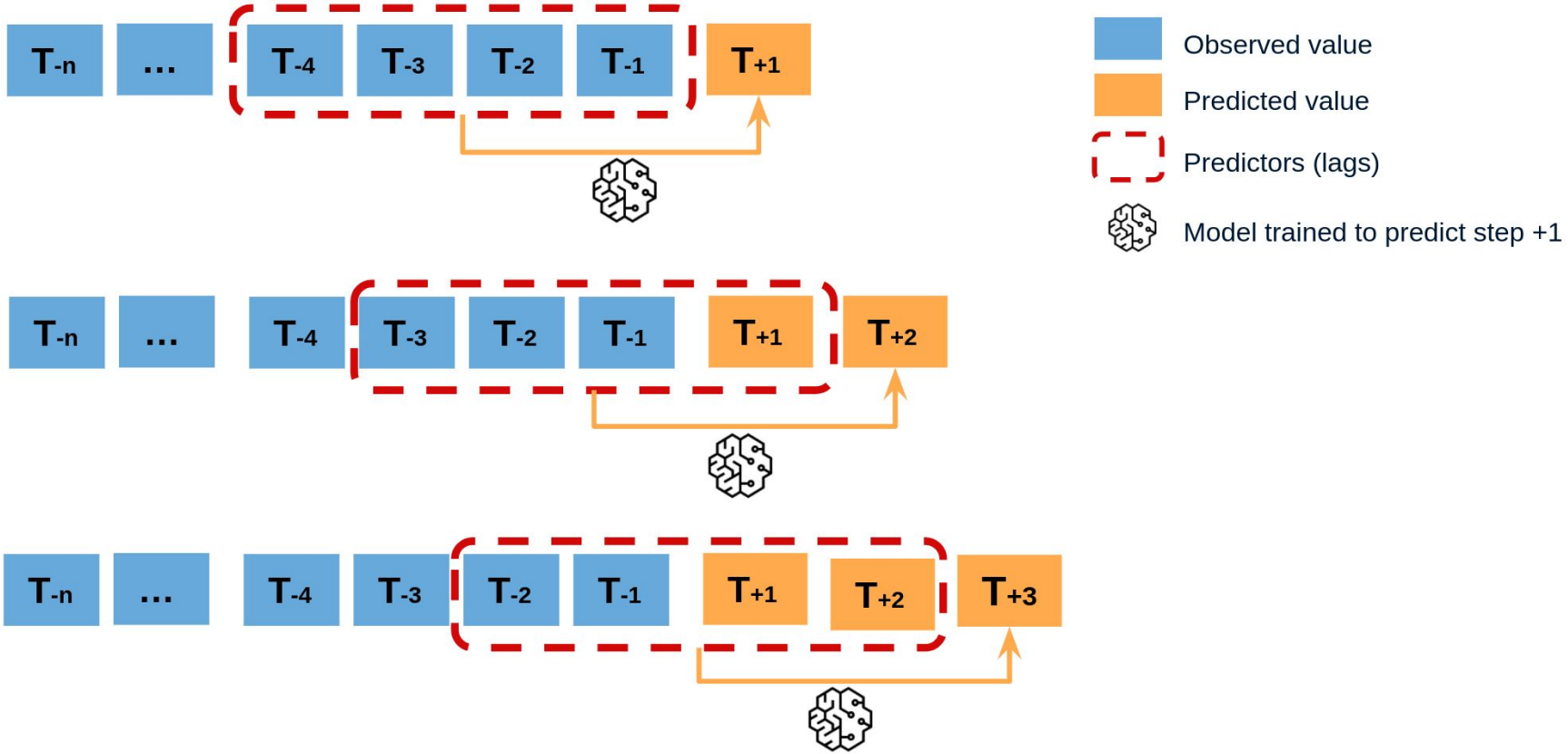
1	2	3	4	5	6	7	8	9	10

Exogenous variable

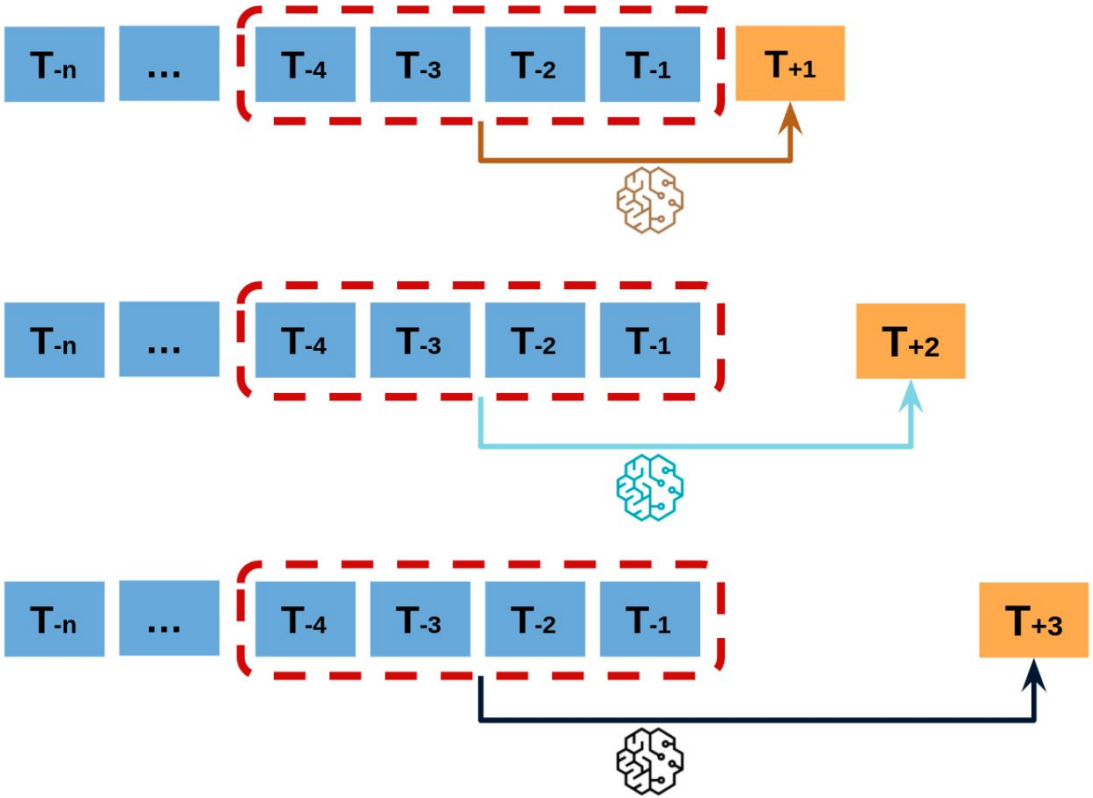
a	b	c	d	e	f	g	h	i	j

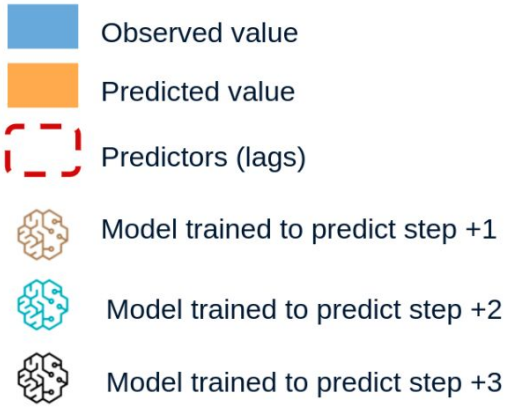
X						y
1	2	3	4	5	f	6
2	3	4	5	6	g	7
3	4	5	6	7	h	8
4	5	6	7	8	i	9
5	6	7	8	9	j	10

Recursive multi-step Time series Forecasting

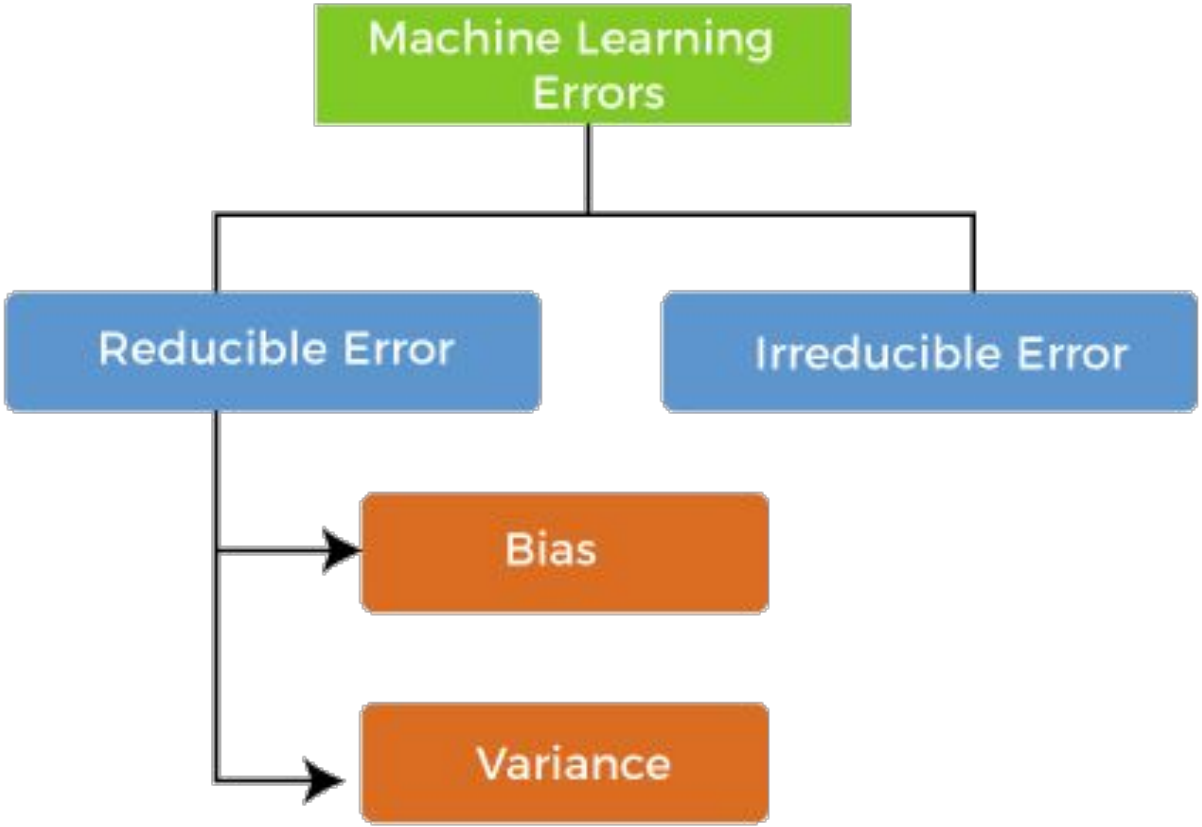


Direct multi-step Time series Forecasting





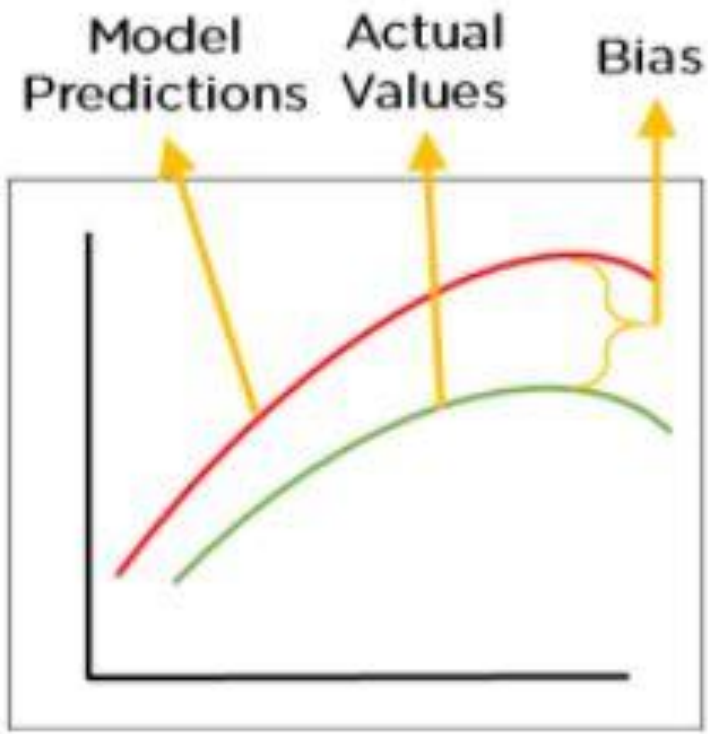
Error in Machine Learning



Error in Machine Learning

Bias

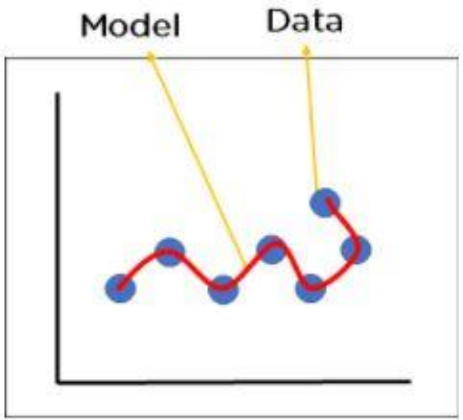
- Sai lệch giữa prediction và actual label
- Cho ta biết khả năng dự đoán chính xác của mô hình
- Bias càng bé càng tốt
- High bias means:
  - Overly-simplified model
  - Under-fitting
  - High error on both training and test sets

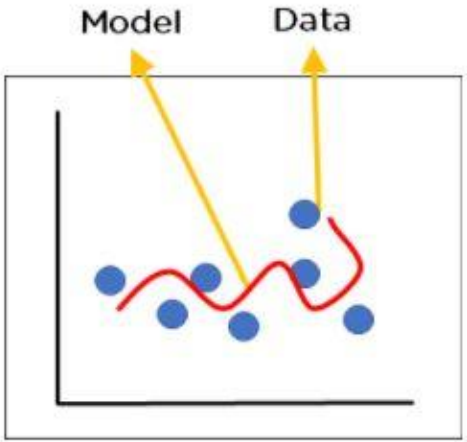


Error in Machine Learning

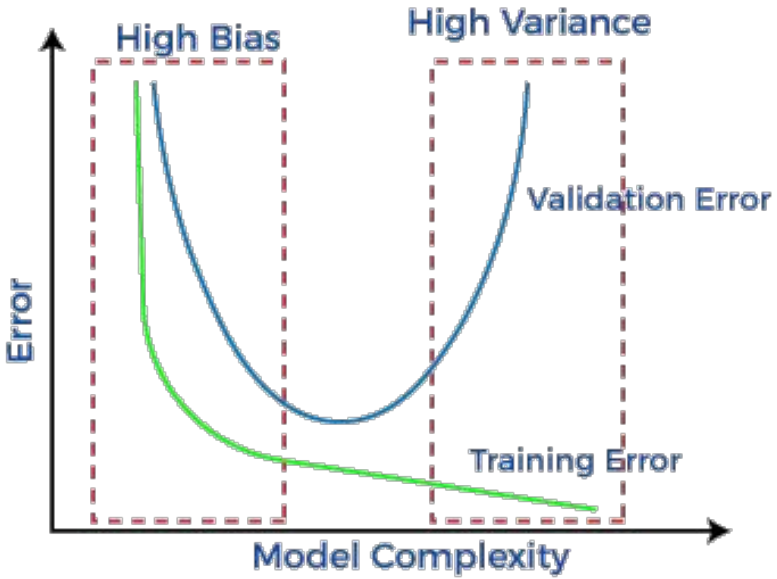
Variance

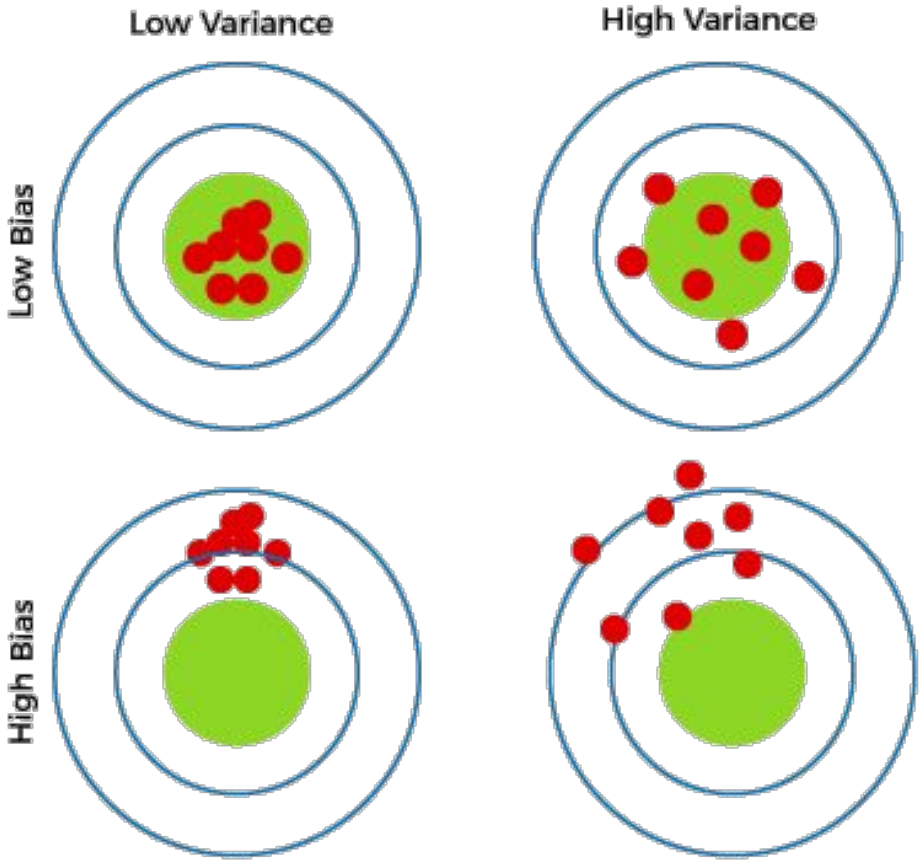
- Mức độ thay đổi của độ chính xác của prediction nếu input thay đổi
- Cho ta biết khả năng tổng quát hóa của mô hình
- Variance càng bé càng tốt
- High variance means:
  - Overly-complex model
  - Over-fitting
  - Low error on training set but high error on test set

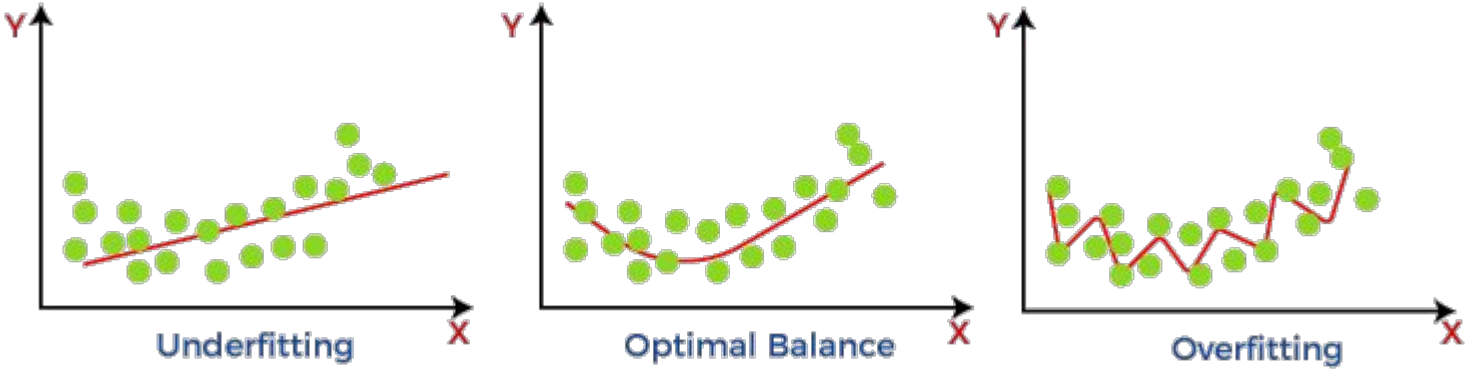


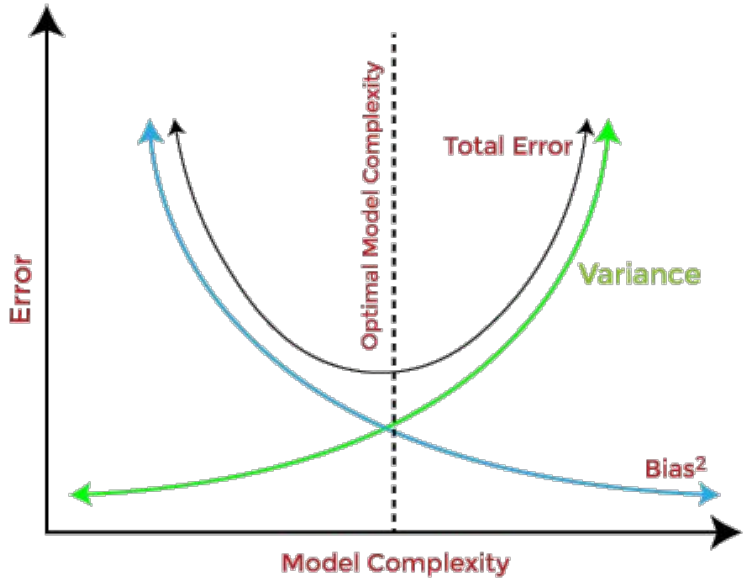


Bias-Variance tradeoff






Y
X
Underfitting
Y
X
Optimal Balance
Y
X
Overfitting


Error
Optimal Model Complexity
Total Error
Variance
Bias²
Model Complexity

Choose Machine Learning model based on data

Low bias + high variance

- Decision tree
- Random Forest
- K-nearest neighbours
- Kernel SVM

-> Phù hợp khi có nhiều data và ít features

High bias + low variance

- Linear regression
- Logistic regression
- Linear SVM

-> Phù hợp khi có ít data và nhiều features

Extra

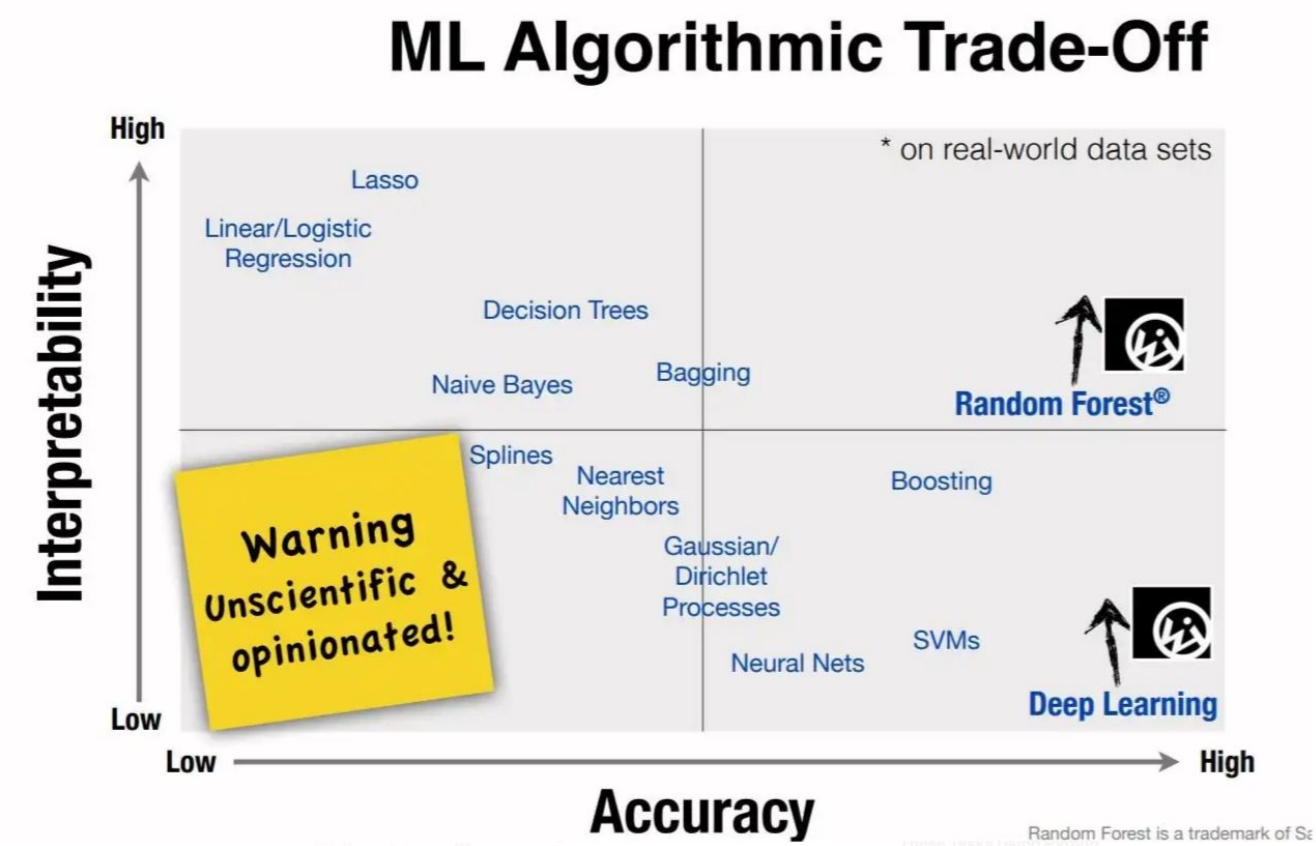
Nếu có nhiều data và nhiều features

- Giảm chiều dữ liệu (e.g. PCA)
- Sử dụng Neural network

Choose Machine Learning model based on business domain

- **Which metric is important?**
  - Accuracy, precision, recall, F1, ...
- **What is the priority?**
  - Speed: self-driving car app need to be realtime -> fast
  - Memory: Model deployed in embedded device needs to be small
  - Accuracy: In medical field, accuracy is the most important factor
- **Interpretability ?**
  - Do you need to explain result?
  - Do you need to find out which features are important?

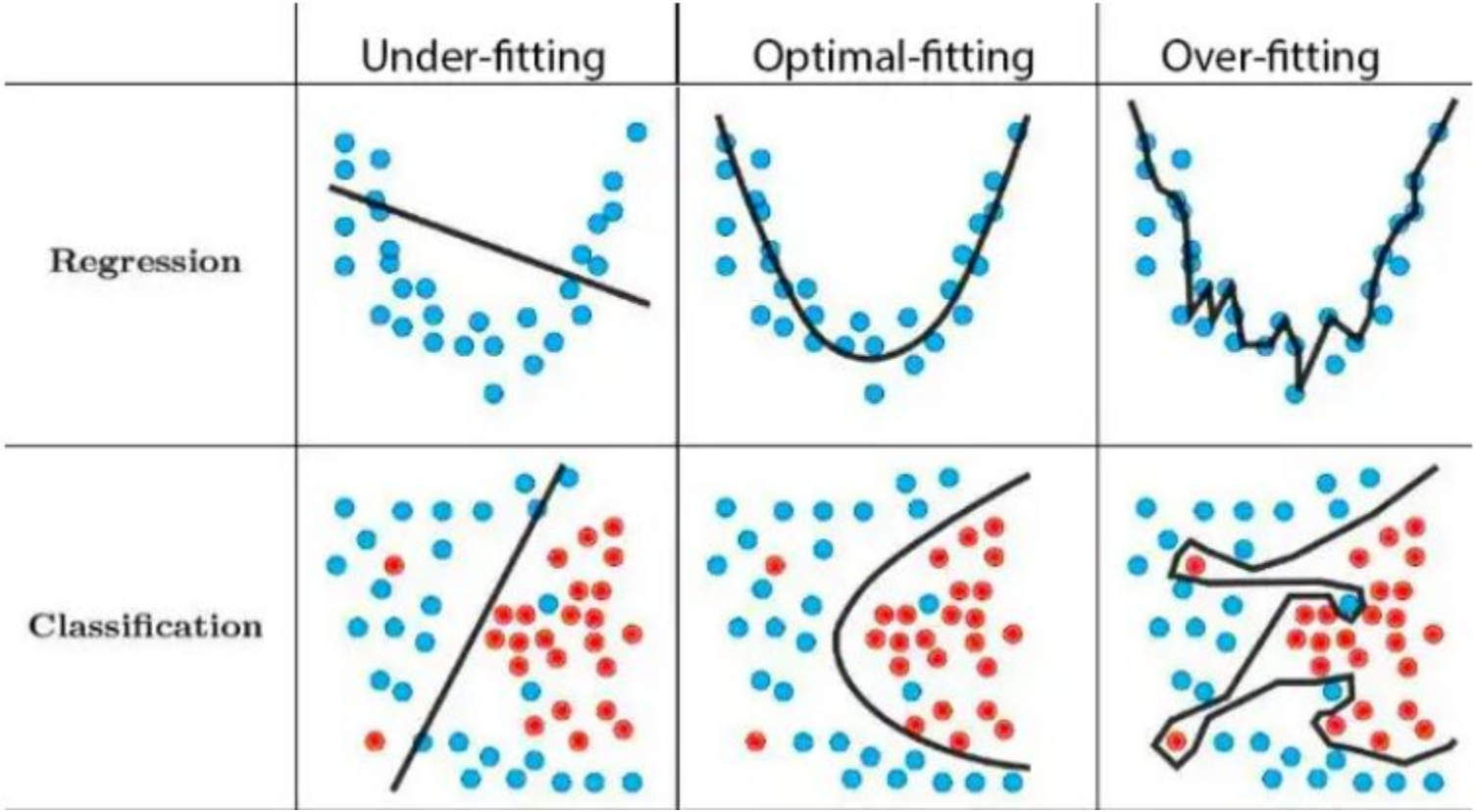
Machine Learning algorithm's interpretability



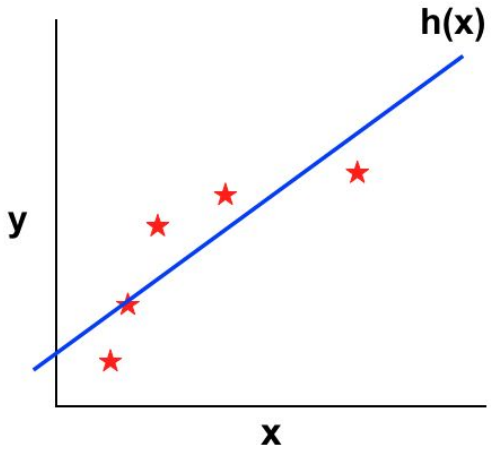
Tips for choosing Machine Learning model

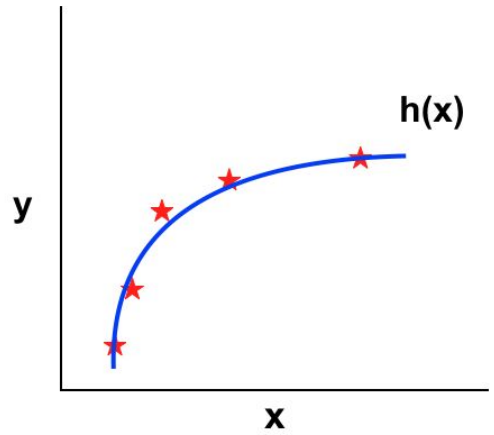
- Start with a simple model
  - Choose the simplest model first
  - If it is good enough, you even do not need to try another model
- Try different models and shortlist the best ones?
- Do Hyperparameter Tuning for each models?
  - GridSearchCV if the number of combinations is small
  - RandomizedSearchCV if the number of combination is large
- Compare amongst the best models with best hyperparameters to pick up the best one

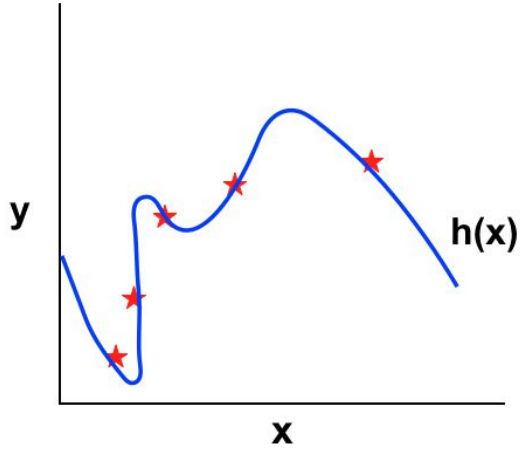
Underfitting and Overfitting



Regularization







$$L(x,y) = \sum_{i=1}^{n} (y_i - f(x_i))^2$$

$$f(x_i) = h_\theta x = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3 + \theta_4 x_4^4$$

$$f(x_i) = h_\theta x = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3 + \theta_4 x_4^4$$

$$f(x_i) = h_\theta x = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2$$

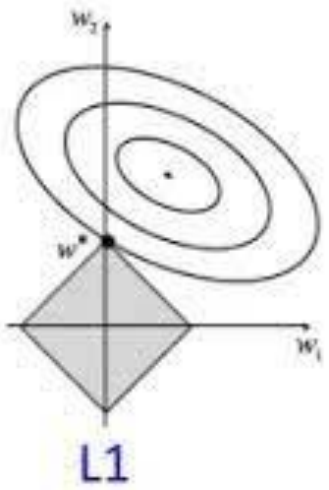
Regularization

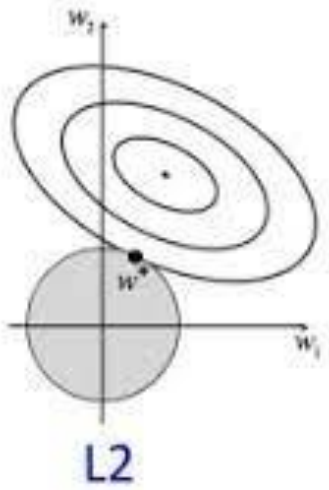
L1 Regularization

$$\text{Cost} = \sum_{i=0}^{N} (y_i - \sum_{j=0}^{M} x_{ij} W_j)^2 + \lambda \sum_{j=0}^{M} |W_j|$$

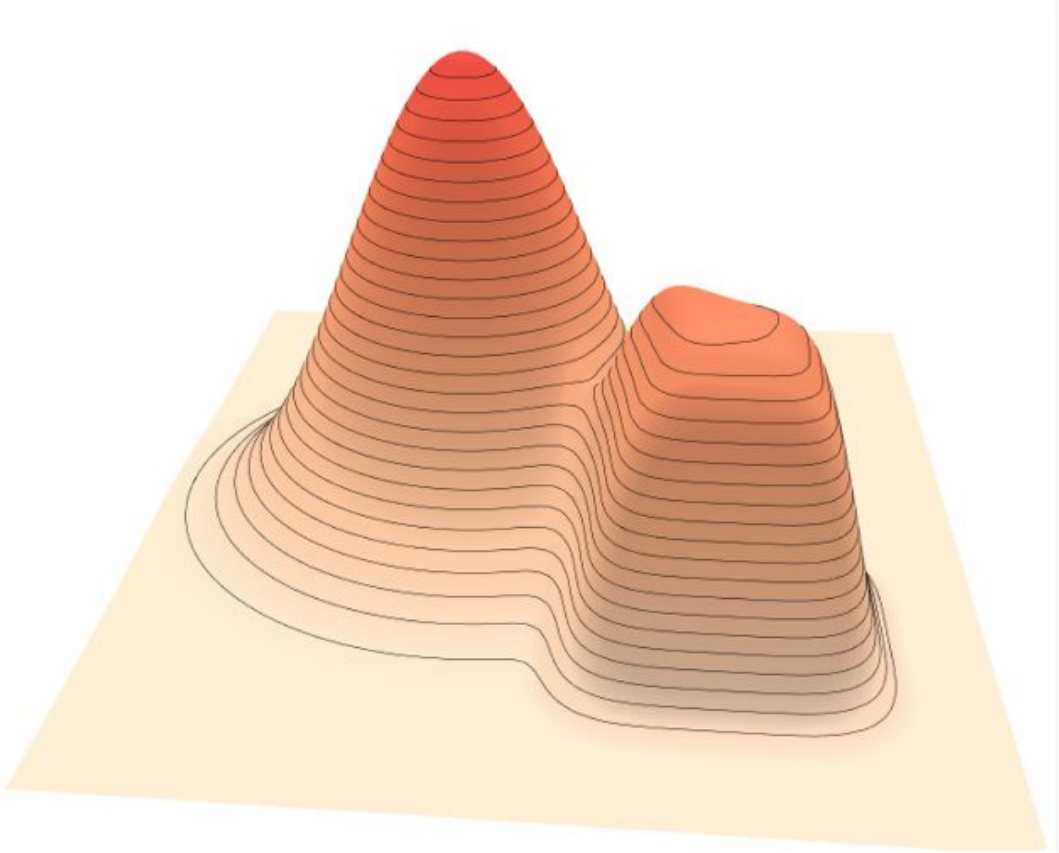
L2 Regularization

$$\text{Cost} = \underbrace{\sum_{i=0}^{N} (y_i - \sum_{j=0}^{M} x_{ij} W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^{M} W_j^2}_{\text{Regularization Term}}$$

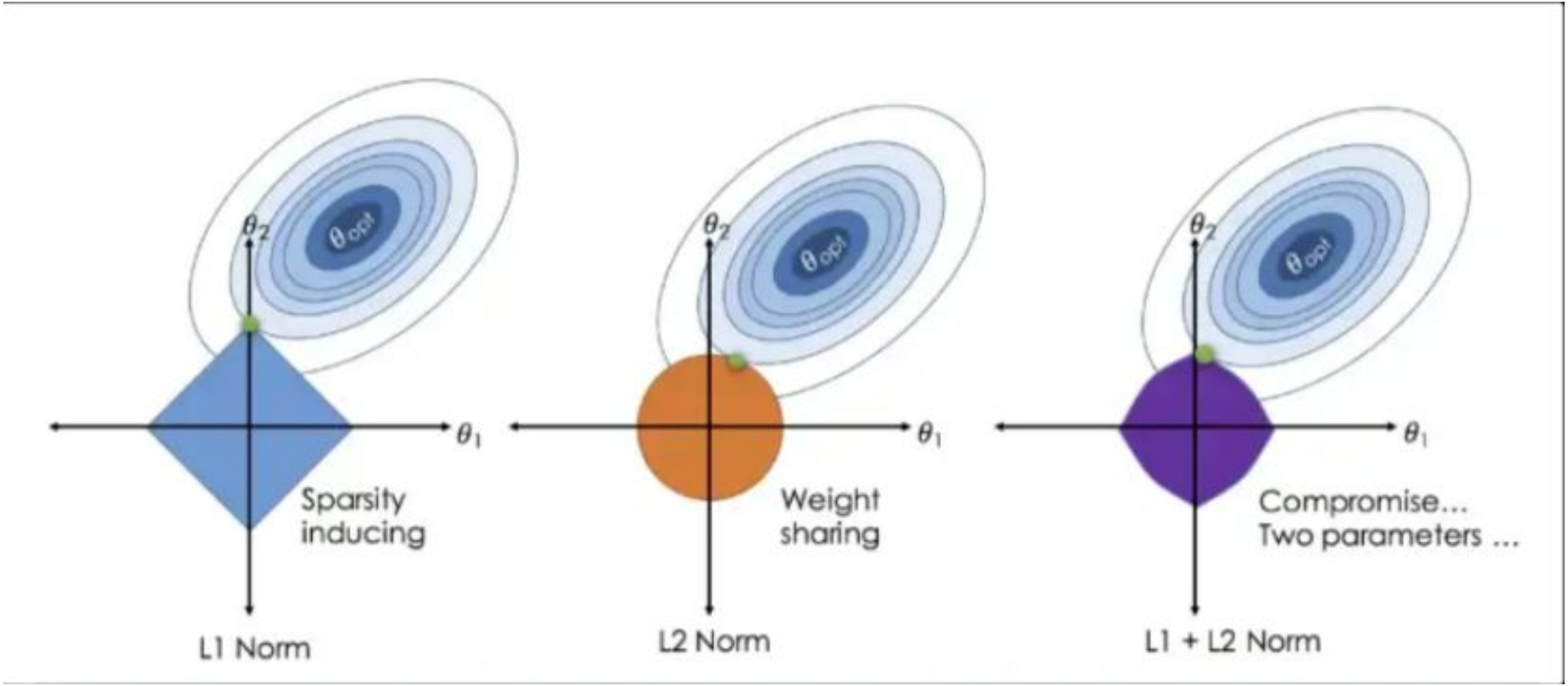




Regularization



Regularization



Regularization Example

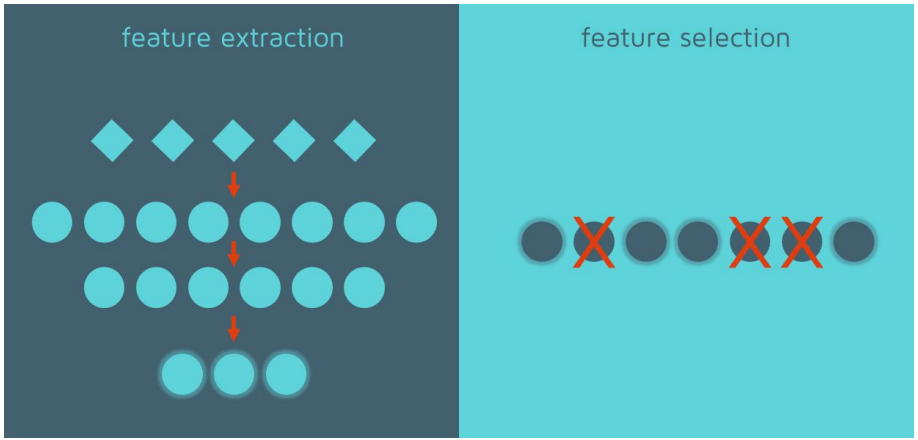
```python
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

data = pd.read_csv("diabetes.csv")
x = data.drop("Outcome", axis=1)
y = data["Outcome"]
cls = Pipeline(steps=[
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(penalty="l1", solver="liblinear"))
])
cls.fit(x, y)
for i, j in zip(x.columns, cls["model"].coef_[0]):
    print("{} is with coefficient of {}".format(i, j))
```

```
fuel_viz
C:\Users\Viet\anaconda3\envs\basic\python.exe C:\Users\Viet\PycharmPr
Pregnancies is with coefficient of 0.4047757715484604
Glucose is with coefficient of 1.1020534150332097
BloodPressure is with coefficient of -0.23905878835657515
SkinThickness is with coefficient of 0.0
Insulin is with coefficient of -0.12014123606887196
BMI is with coefficient of 0.6883338771775394
DiabetesPedigreeFunction is with coefficient of 0.30157785094254597
Age is with coefficient of 0.16766878675897076
```

```python
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

data = pd.read_csv("diabetes.csv")
x = data.drop("Outcome", axis=1)
y = data["Outcome"]
cls = Pipeline(steps=[
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(penalty="l2", solver="lbfgs"))
])
cls.fit(x, y)
for i, j in zip(x.columns, cls["model"].coef_[0]):
    print("{} is with coefficient of {}".format(i, j))
```

```
fuel_viz
C:\Users\Viet\anaconda3\envs\basic\python.exe C:\Users\Viet\PycharmP
Pregnancies is with coefficient of 0.4086468712956544
Glucose is with coefficient of 1.107111971571363
BloodPressure is with coefficient of -0.2508680388517632
SkinThickness is with coefficient of 0.00905045508333392
Insulin is with coefficient of -0.13083627394872085
BMI is with coefficient of 0.6963087173103848
DiabetesPedigreeFunction is with coefficient of 0.3088372414180195
Age is with coefficient of 0.1764978217442397
```

Dimensionality Reduction



01

Feature Selection

Original features are maintained

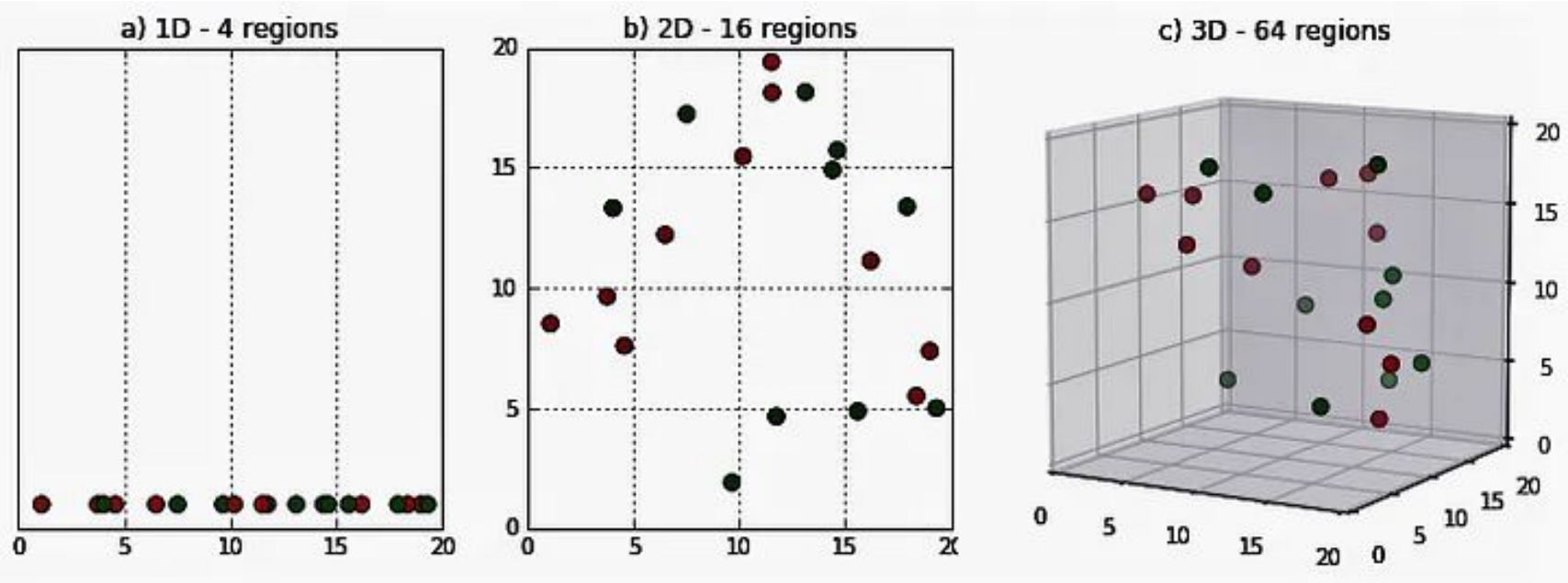
Keep most important features

02

Feature Extraction

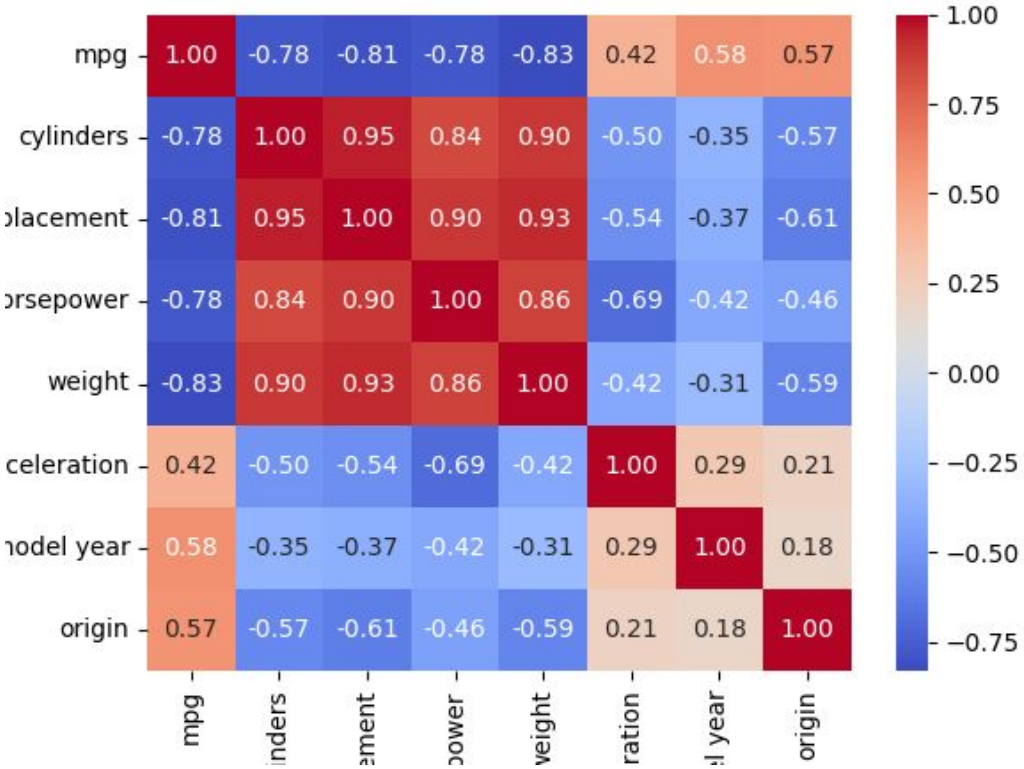
Features are transformed to a new space

Curse of Dimensionality



Feature Selection

Feature Selection - Correlation Coefficient



Feature Selection - Variance Threshold

```python
import pandas as pd
from sklearn.feature_selection import VarianceThreshold

data = pd.read_csv("diabetes.csv", usecols=[i for i in range(8)])
variance_threshold = VarianceThreshold(threshold=1)
variance_threshold.fit(data)
print(variance_threshold.variances_)
print(variance_threshold.get_support())
```

fuel_viz

```
C:\Users\Viet\anaconda3\envs\basic\python.exe C:\Users\Viet\Pycharm
[1.13392724e+01 1.02091726e+03 3.74159449e+02 2.54141900e+02
 1.32638869e+04 6.20790465e+01 1.09635697e-01 1.38122964e+02]
[ True  True  True  True  True  True False  True]
```

Feature Selection - Lasso (L1)

```python
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

data = pd.read_csv("diabetes.csv")
x = data.drop("Outcome", axis=1)
y = data["Outcome"]
cls = Pipeline(steps=[
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(penalty="l1", solver="liblinear"))
])
cls.fit(x, y)
for i, j in zip(x.columns, cls["model"].coef_[0]):
    print("{} is with coefficient of {}".format(i, j))
```

```
fuel_viz
C:\Users\Viet\anaconda3\envs\basic\python.exe C:\Users\Viet\PycharmPr
Pregnancies is with coefficient of 0.4047757715484604
Glucose is with coefficient of 1.1020534150332097
BloodPressure is with coefficient of -0.23905878835657515
SkinThickness is with coefficient of 0.0
Insulin is with coefficient of -0.12014123606887196
BMI is with coefficient of 0.6883338771775394
DiabetesPedigreeFunction is with coefficient of 0.30157785094254597
Age is with coefficient of 0.16766878675897076
```

```python
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

data = pd.read_csv("diabetes.csv")
x = data.drop("Outcome", axis=1)
y = data["Outcome"]
cls = Pipeline(steps=[
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(penalty="l2", solver="lbfgs"))
])
cls.fit(x, y)
for i, j in zip(x.columns, cls["model"].coef_[0]):
    print("{} is with coefficient of {}".format(i, j))
```

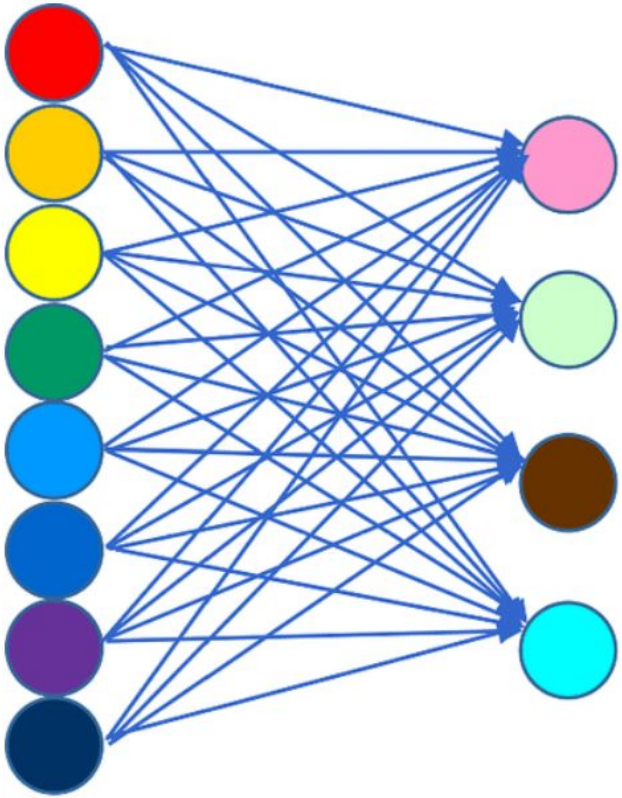
```
fuel_viz
C:\Users\Viet\anaconda3\envs\basic\python.exe C:\Users\Viet\PycharmP
Pregnancies is with coefficient of 0.4086468712956544
Glucose is with coefficient of 1.107111971571363
BloodPressure is with coefficient of -0.2508680388517632
SkinThickness is with coefficient of 0.00905045508333392
Insulin is with coefficient of -0.13083627394872085
BMI is with coefficient of 0.6963087173103848
DiabetesPedigreeFunction is with coefficient of 0.3088372414180195
Age is with coefficient of 0.1764978217442397
```

Feature Selection - RandomForest

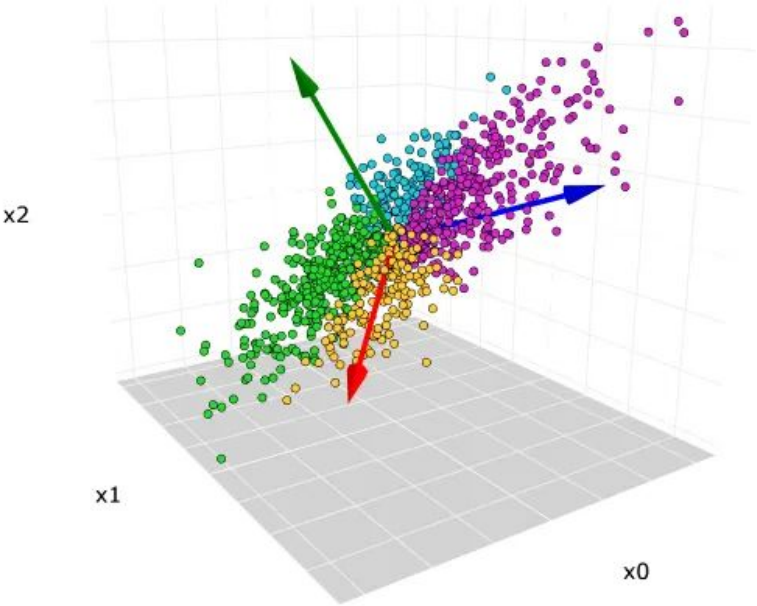
```python
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

data = pd.read_csv("diabetes.csv")
x = data.drop("Outcome", axis=1)
y = data["Outcome"]
cls = Pipeline(steps=[
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier())
])
cls.fit(x, y)
print(cls["model"].feature_importances_)
```

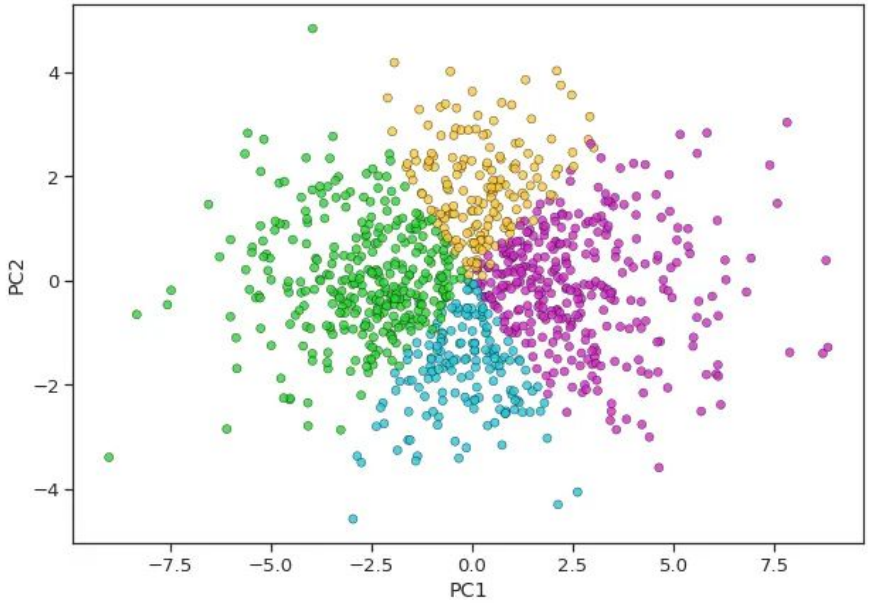
```
fuel_viz
C:\Users\Viet\anaconda3\envs\basic\python.exe C:\Users\Viet\Pycharm
[0.08485069 0.26544363 0.09088871 0.07260613 0.07047188 0.15482689
 0.12826565 0.13264643]
```

Feature Extraction



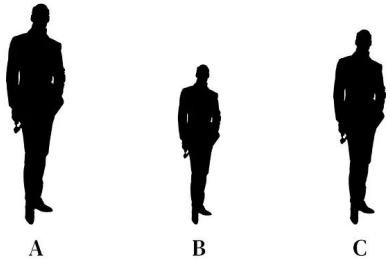
Feature Extraction - Principle Component Analyst



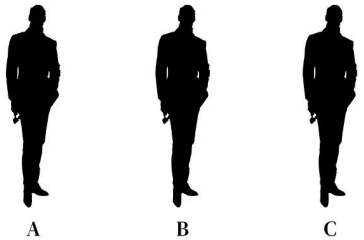


Variance in data

$$\begin{aligned}\operatorname{Var}(X) &= \mathrm{E}\left[(X - \mathrm{E}[X])^2\right] \\ &= \mathrm{E}\left[X^2 - 2X\,\mathrm{E}[X] + \mathrm{E}[X]^2\right] \\ &= \mathrm{E}\left[X^2\right] - 2\,\mathrm{E}[X]\,\mathrm{E}[X] + \mathrm{E}[X]^2 \\ &= \mathrm{E}\left[X^2\right] - \mathrm{E}[X]^2\end{aligned}$$



Person	Height (cm)
Alex	145
Ben	160
Chris	185



Person	Height (cm)
Daniel	172
Elsa	173
Fernandez	171

Covariance vs Correlation

$$\begin{aligned}\operatorname{Var}(X) &= \mathrm{E}\left[(X - \mathrm{E}[X])^2\right] \\ &= \mathrm{E}\left[X^2 - 2X\,\mathrm{E}[X] + \mathrm{E}[X]^2\right] \\ &= \mathrm{E}\left[X^2\right] - 2\,\mathrm{E}[X]\,\mathrm{E}[X] + \mathrm{E}[X]^2 \\ &= \mathrm{E}\left[X^2\right] - \mathrm{E}[X]^2\end{aligned}$$

$$\begin{aligned}\operatorname{cov}(X, Y) &= \mathrm{E}[(X - \mathrm{E}[X])\,(Y - \mathrm{E}[Y])] \\ &= \mathrm{E}[XY - X\,\mathrm{E}[Y] - \mathrm{E}[X]Y + \mathrm{E}[X]\,\mathrm{E}[Y]] \\ &= \mathrm{E}[XY] - \mathrm{E}[X]\,\mathrm{E}[Y] - \mathrm{E}[X]\,\mathrm{E}[Y] + \mathrm{E}[X]\,\mathrm{E}[Y] \\ &= \mathrm{E}[XY] - \mathrm{E}[X]\,\mathrm{E}[Y],\end{aligned}$$

$$\mathrm{Corr}(X,Y) = \frac{\mathrm{Cov}(X,Y)}{\sigma_x \sigma_y}$$

Covarianced normalized by Standard Deviation

Correlation between X and Y

Standard deviation of X

Standard deviation of Y

$$\textbf{Standard Deviation} = \sqrt{\boldsymbol{Variance}}$$

Covariance vs Correlation

Covariance	Correlation
Indicates the direction of the linear relationship between variables	Indicates both the strength and direction of the linear relationship between two variables
Covariance values are not standard	Correlation values are standardized
Positive number being positive relationship and negative number being negative relationship	1 being strong positive correlation, -1 being strong negative correlation
Value between positive infinity to negative infinity	Value is strictly between -1 to 1

PCA: step 1 - Standardize the dataset

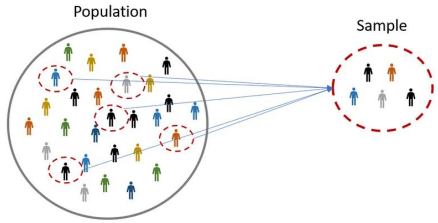
f1	f2	f3	f4
1	2	3	4
5	5	6	7
1	4	2	3
5	3	2	1
8	1	2	2

$$x_{new} = \frac{x - \mu}{\sigma}$$

		f1	f2	f3	f4
μ	=	4	3	3	3.4
σ	=	3	1.58114	1.73205	2.30217

f1	f2	f3	f4
-1	-0.63246	0	0.26062
0.33333	1.26491	1.73205	1.56374
-1	0.63246	-0.57735	-0.17375
0.33333	0	-0.57735	-1.04249
1.33333	-1.26491	-0.57735	-0.60812

PCA: step 2 - Calculate the covariance matrix



f1	f2	f3	f4
-1	-0.63246	0	0.26062
0.33333	1.26491	1.73205	1.56374
-1	0.63246	-0.57735	-0.17375
0.33333	0	-0.57735	-1.04249
1.33333	-1.26491	-0.57735	-0.60812

For Population

$$Cov(x,y) = \frac{\Sigma (x_i - \bar{x}) * (y_i - \bar{y})}{N}$$

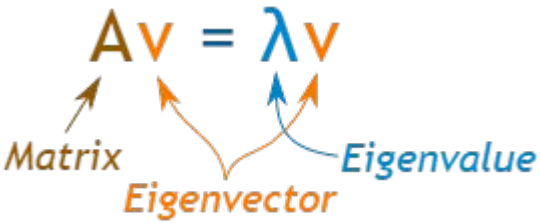
For Sample

$$Cov(x,y) = \frac{\Sigma (x_i - \bar{x}) * (y_i - \bar{y})}{(N-1)}$$

	f1	f2	f3	f4
f1	var(f1)	cov(f1,f2)	cov(f1,f3)	cov(f1,f4)
f2	cov(f2,f1)	var(f2)	cov(f2,f3)	cov(f2,f4)
f3	cov(f3,f1)	cov(f3,f2)	var(f3)	cov(f3,f4)
f4	cov(f4,f1)	cov(f4,f2)	cov(f4,f3)	var(f4)

	f1	f2	f3	f4
f1	0.8	-0.25298	0.03849	-0.14479
f2	-0.25298	0.8	0.51121	0.4945
f3	0.03849	0.51121	0.8	0.75236
f4	-0.14479	0.4945	0.75236	0.8

PCA: step 3 - Calculate eigenvector and eigenvalue



Definition. Let A be an $n \times n$ matrix.

1. An ***eigenvector*** of A is a *nonzero* vector v in R^n such that $Av = \lambda v$, for some scalar λ.
2. An ***eigenvalue*** of A is a scalar λ such that the equation $Av = \lambda v$ has a *nontrivial* solution.

If $Av = \lambda v$ for $v \neq 0$, we say that λ is the ***eigenvalue for*** v, and that v is an ***eigenvector for*** λ.

$$A = \begin{bmatrix} 1 & 4 \\ 3 & 2 \end{bmatrix}$$

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

$$\det(A - \lambda I) = 0$$

$$\det\left(\begin{bmatrix} 1 & 4 \\ 3 & 2 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\right) = 0$$

$$\det\left(\begin{bmatrix} 1-\lambda & 4 \\ 3 & 2-\lambda \end{bmatrix}\right) = 0$$

$$(1-\lambda)(2-\lambda) - 12 = 0$$

$$\lambda^2 - 3\lambda - 10 = 0$$

$$(\lambda - 5)(\lambda + 2) = 0$$

$$\lambda = 5, -2$$

PCA: step 3 - Calculate eigenvector and eigenvalue

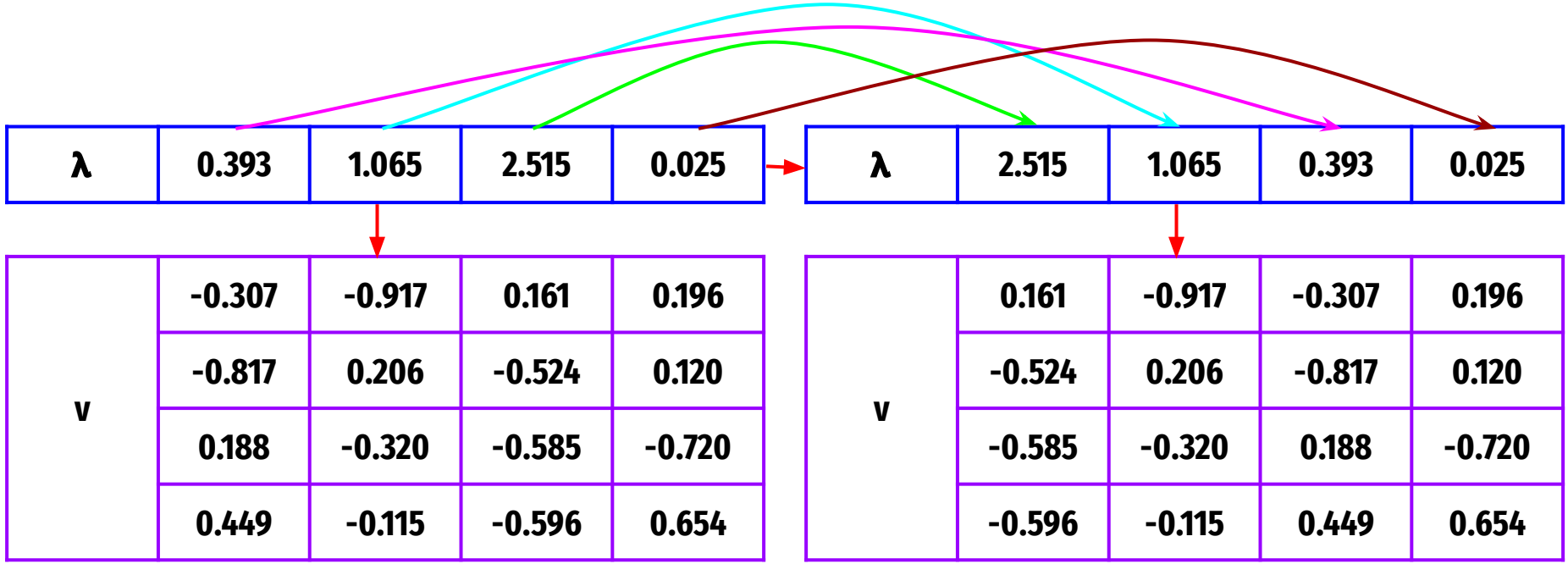
	f1	f2	f3	f4
f1	0.8	-0.25298	0.03849	-0.14479
f2	-0.25298	0.8	0.51121	0.4945
f3	0.03849	0.51121	0.8	0.75236
f4	-0.14479	0.4945	0.75236	0.8

	f1	f2	f3	f4
f1	0.8 - λ	-0.25298	0.03849	-0.14479
f2	-0.25298	0.8- λ	0.51121	0.4945
f3	0.03849	0.51121	0.8 - λ	0.75236
f4	-0.14479	0.4945	0.75236	0.8 - λ

λ	**0.393**	**1.065**	**2.515**	**0.025**

v	**-0.307**	**-0.917**	**0.161**	**0.196**
	-0.817	**0.206**	**-0.524**	**0.120**
	0.188	**-0.320**	**-0.585**	**-0.720**
	0.449	**-0.115**	**-0.596**	**0.654**

PCA: step 4 - Sort eigenvalues & corresponding eigenvectors



PCA: step 5 - Pick k eigenvalues and form a matrix

k=2

λ	2.515	1.065	~~0.393~~	~~0.025~~

v	0.161	-0.917
	-0.524	0.206
	-0.585	-0.320
	-0.596	-0.115

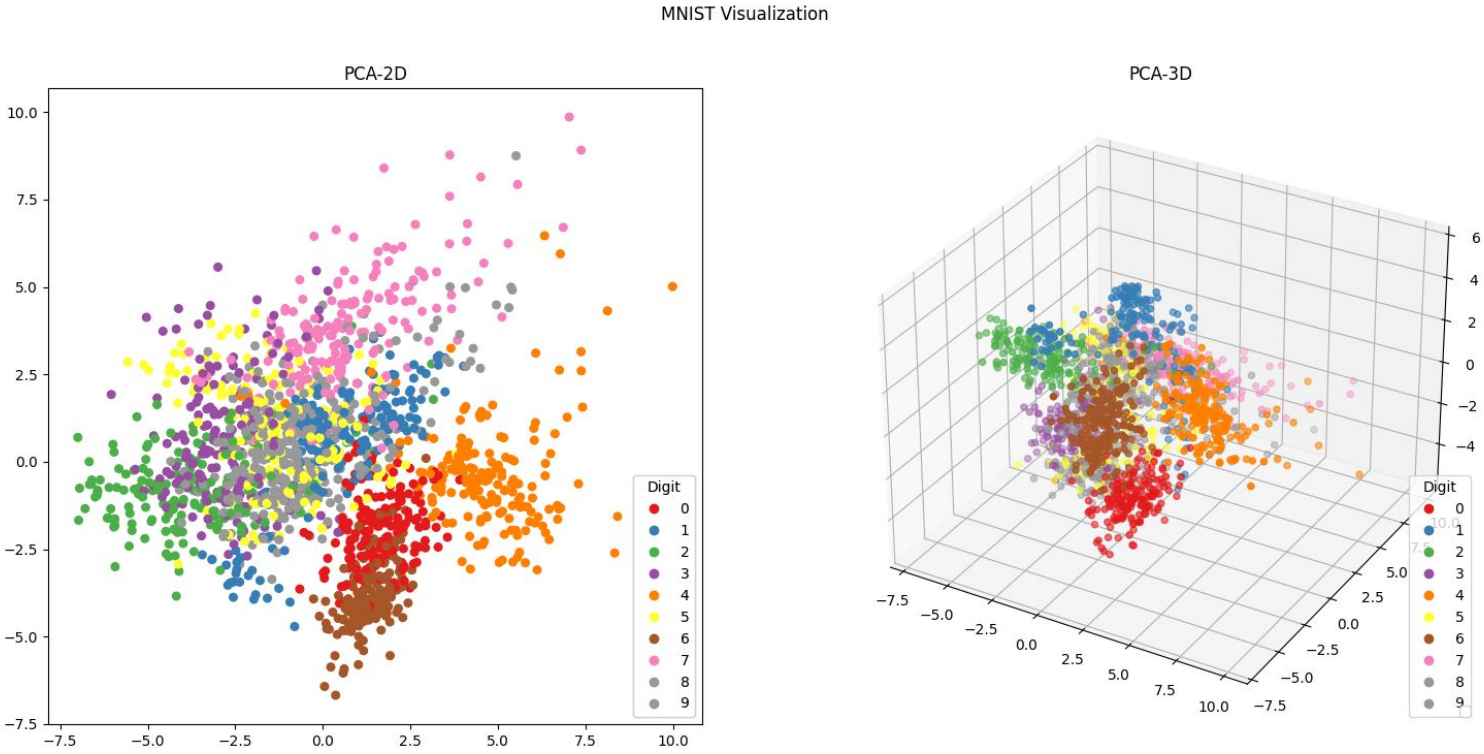
PCA: step 6 - Transform feature matrix

Nx4 matrix x

0.161	-0.917
-0.524	0.206
-0.585	-0.320
-0.596	-0.115

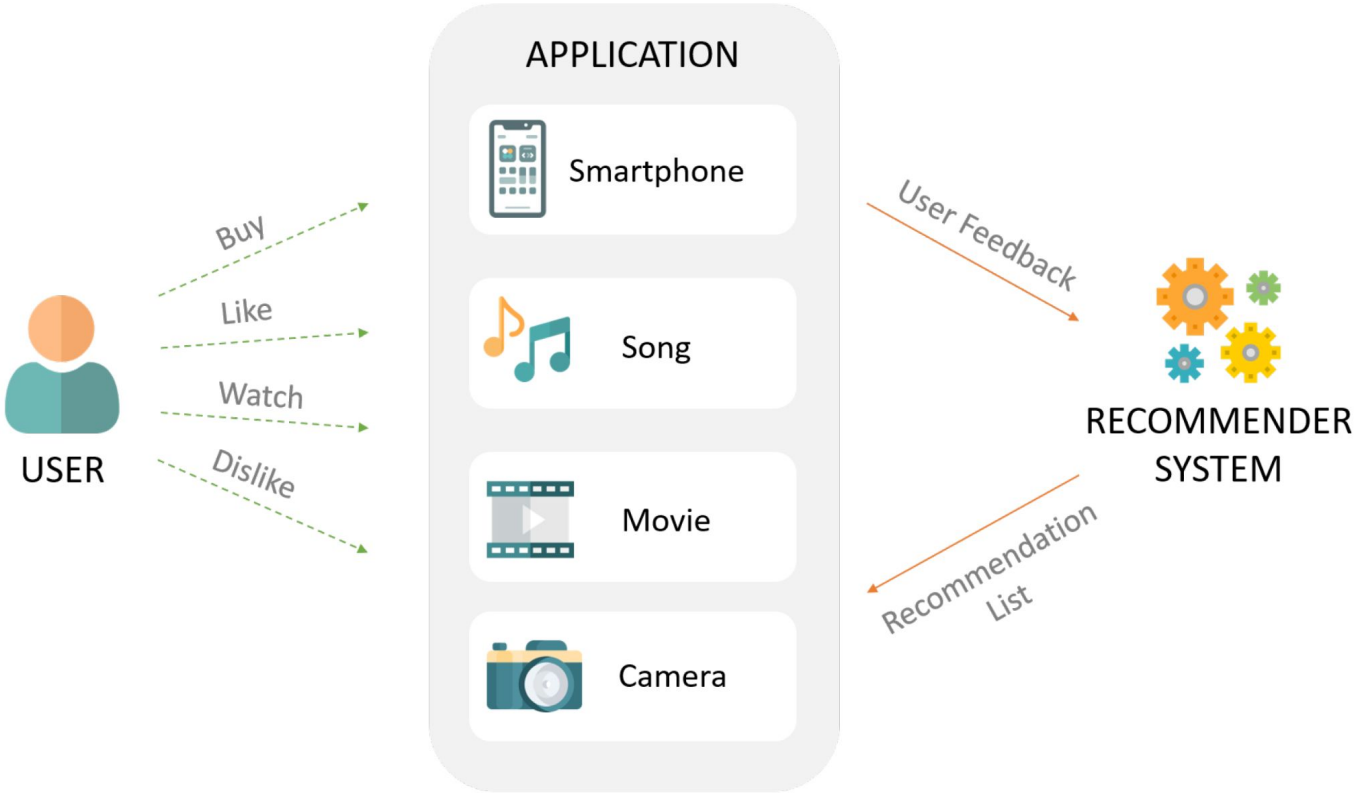
= Nx2 matrix

PCA visualization

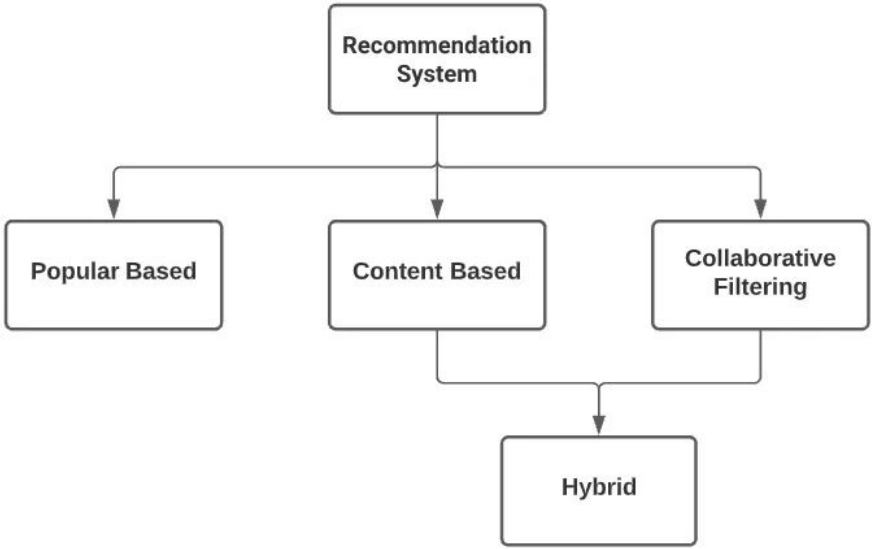


Recommendation systems

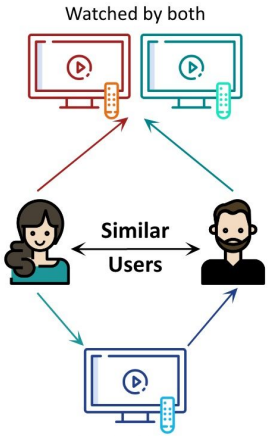
What is recommendation systems



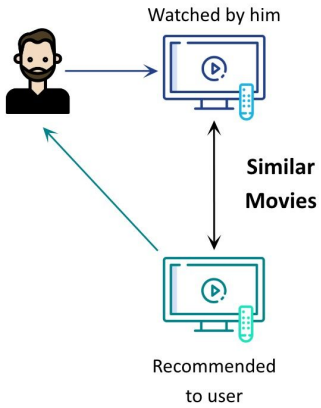
Types of recommendation systems



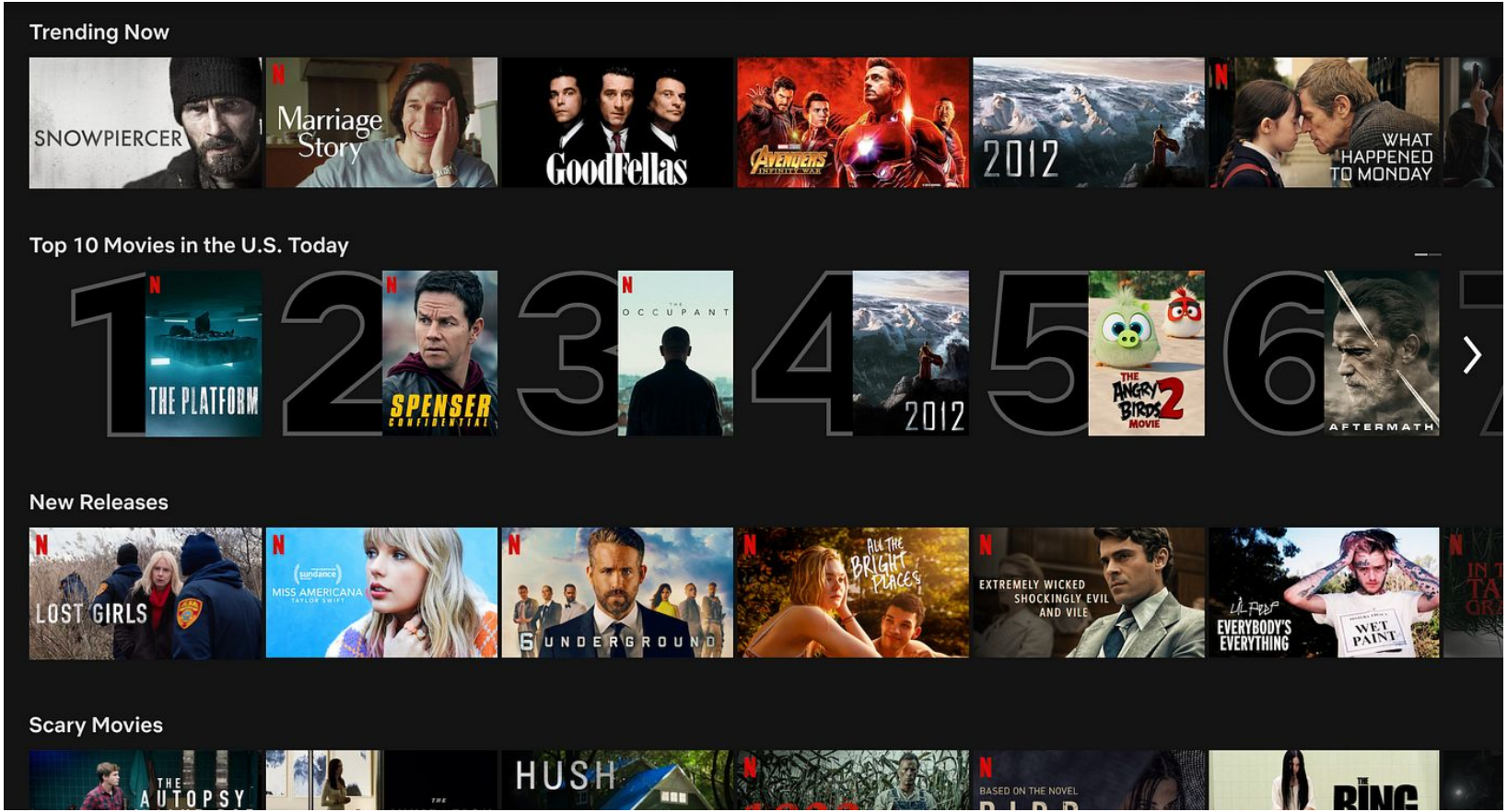
Collaborative Filtering



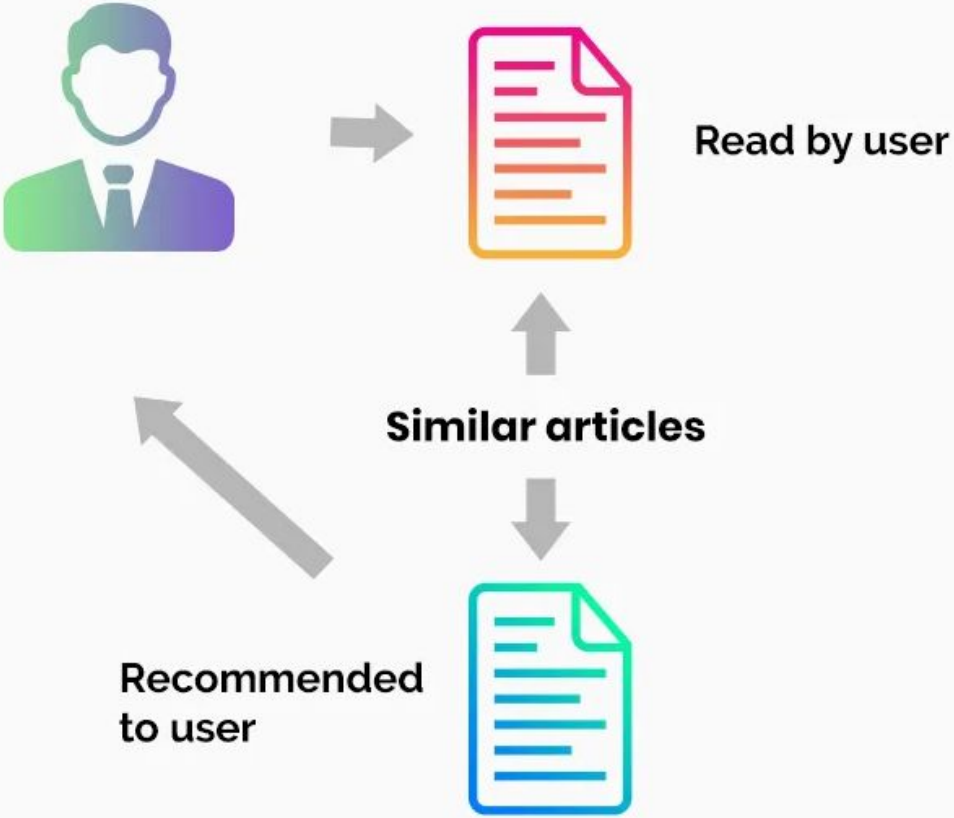
Content-Based Filtering



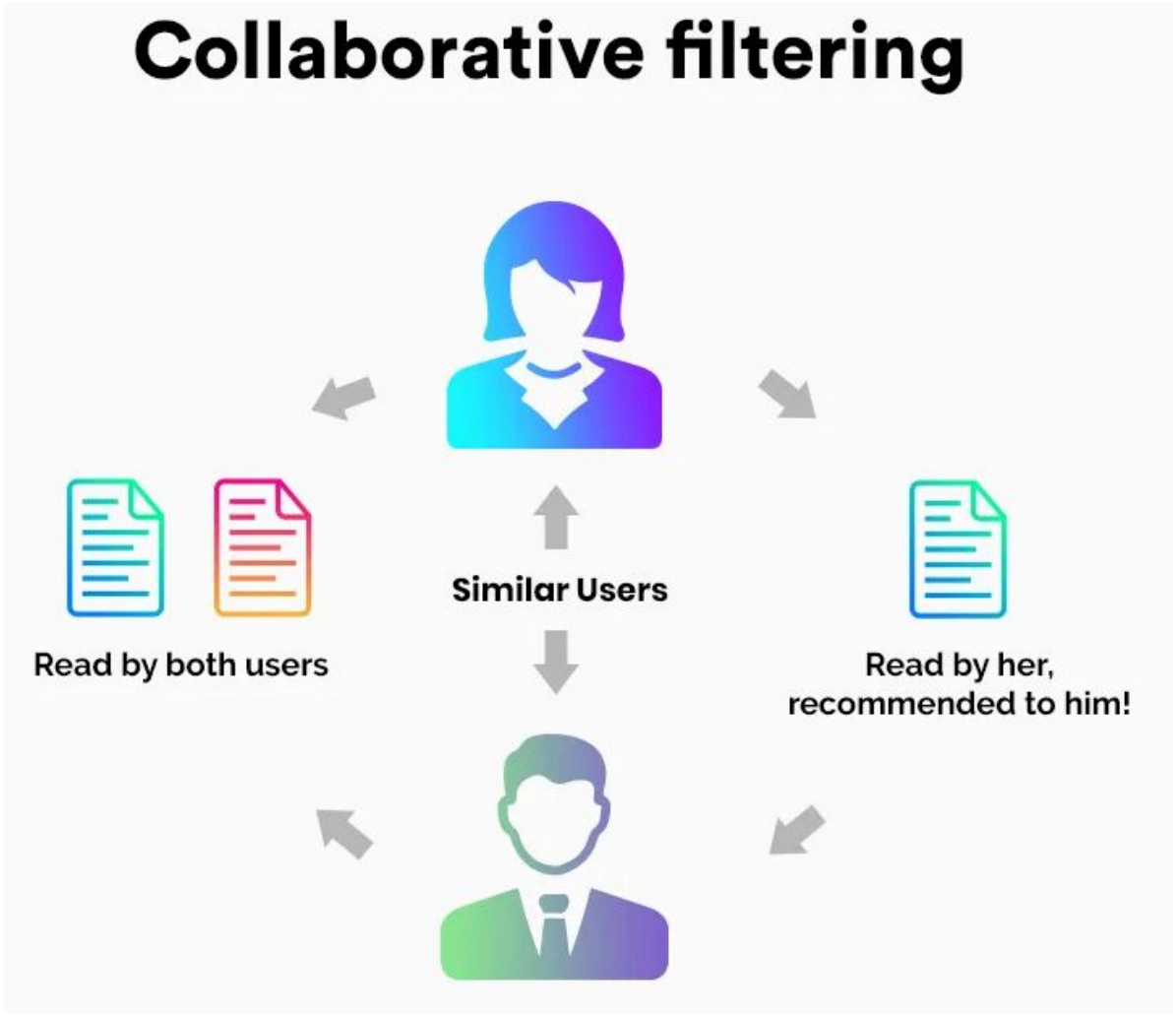
Popularity-based recommendation systems



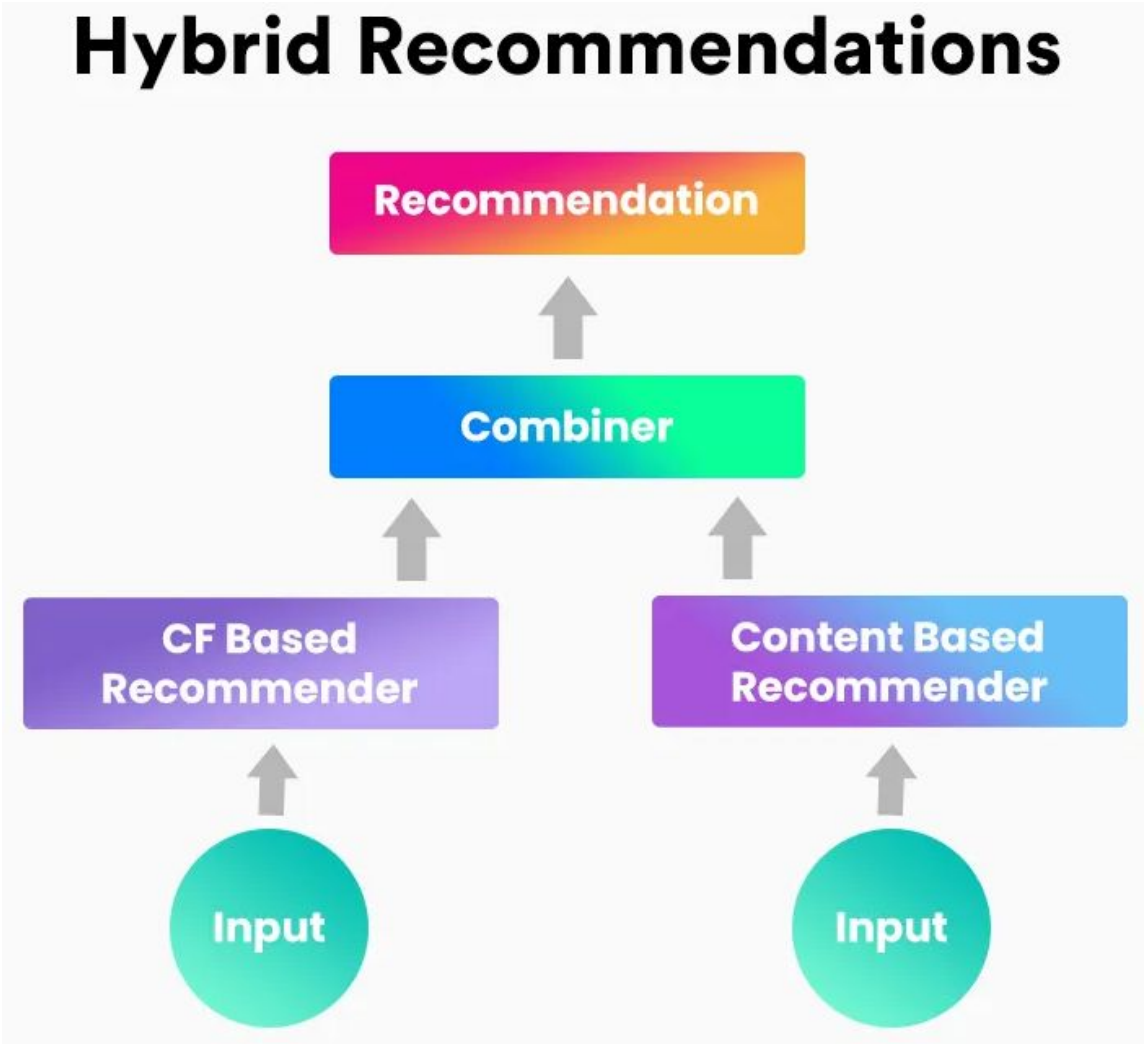
Content-based filtering



Collaborative filtering



Hybrid Recommendations



Utility matrix